

Integer Commitments, Old and New Tools

Iftach Haitner*

Yehuda Lindell†

Nikolaos Makriyannis‡

March 17, 2026

Abstract

This self-contained and detailed tutorial covers RSA-based integer commitments and related protocols. It also presents a new, highly efficient setup protocol for sampling commitment parameters.

Contents

1	Introduction	3
1.1	Our Contribution	3
1.2	Related work on Range Proofs	5
2	Preliminaries	6
2.1	Notation	6
2.2	Sigma Protocols	6
2.3	Quadratic Residuosity for Primes and Biprimes	8
2.3.1	Basic Notation	8
2.3.2	Jacobi Symbol	8
2.3.3	The Group QR_n	10
2.3.4	Structure of $\mathbb{Z}_n^*$	11
2.4	The Strong RSA Assumption	12
2.4.1	Two Hard Games	12
3	The Integer Commitment Scheme	15
3.1	Security	15
3.2	Parameters Generation	17
4	Range Proof based on Strong RSA	18

*Stellar Development Foundation and The Blavatnik School of Computer Science, Tel Aviv University. Research supported by Israel Science Foundation grants 836/23. E-mail: iftachh@gmail.com. Part of this work was conducted while at Coinbase.

†Coinbase. E-mail: yehuda.lindell@coinbase.com.

‡Fireblocks. E-mail: nikos@fireblocks.com.

5	Applications to Other Additively Homomorphic Commitments	21
5.1	Equality Proofs	22
5.1.1	Equality between Group Commitments in Rough Groups	22
5.1.2	Equality with Pedersen Commitments in Known-Order Rough Groups	23
5.1.3	Equality with Paillier Encryption	24
5.1.4	Equality between Different Homomorphic Commitments	27
5.2	Range Proofs	27
5.3	Applications	27
5.3.1	Oblivious Linear Evaluation	27
5.3.2	Commit&Modular-Reveal	28
6	Lightweight Leaky Well-Formedness Proof	28
6.1	Applications	30
6.1.1	Commitments over Rough Integer Rings	30
6.1.2	Oblivious Linear Evaluation	31
6.1.3	Commit & Modular Reveal	31
7	Vector Commitments	31
7.1	Lightweight Leaky Well-Formedness Proof for Vector Commitments	32
8	Historical notes	34

1 Introduction

(RSA-based) integer commitments, introduced by Fujisaki and Okamoto [FO97] and Damgård and Fujisaki [DF02], are Pedersen-like additively homomorphic commitments that are binding over the integers (in contrast to Pedersen commitments, which are binding only over the underlying finite ring or field). A key feature of integer commitments is that they admit highly efficient *range proofs*, i.e., protocols for proving knowledge of an opening to a value lying in a prescribed interval. The availability of such efficient range proofs is what makes integer commitments particularly useful in practice. In particular, they serve as a building block for efficient range proofs for other additively homomorphic commitments, including Pedersen and ElGamal commitments over known-order groups, as well as commitments induced by the Paillier public-key encryption scheme. These range proofs can then be leveraged to obtain *equality proofs* which enable proving consistency of the committed values across different commitment/encryption types, e.g., between ElGamal and Paillier. Although integer commitments are not post-quantum secure, they remain widely used when efficiency is paramount—for example, as a component in highly efficient protocols for distributed ECDSA signing, e.g., [GG18; Can+20], which are among the most practically deployed multi-party computation protocols.

1

1.1 Our Contribution

Detailed overview of known facts. In the main part of this paper, we give an overview of the basic protocols and facts about integer commitments and their related tools, starting with the standard hiding and binding properties, and then move to a strong-RSA-based range-proof, with slackness, protocol (see Section 1.2 for other range-proof protocols for integer commitments). Stating the security of the range-proof protocol in a standalone manner is somewhat subtle, since it requires the integer commitment parameters to be properly sampled. For that, we put forward the notion of *computational Sigma protocols secure with respect to a distribution ensemble*, and give a full proof that the range-proof protocol is secure in this sense.

Integer commitments are rarely used for their own sake. Rather, they are typically used for equipping other additively homomorphic commitment schemes, e.g., ElGamal, with an efficient range proof, and to provide efficient equality proofs between different homomorphic commitments. We describe the above tools in detail, and also describe some easy applications of the above.

While none of the above protocols and facts is new, and we provide the relevant references in Section 8, we made the effort to make the security statements standalone and the proofs rigorous and self-contained. In some cases, our proofs are also sharper, yielding somewhat stronger security guarantees than previous ones.

New lightweight setup protocol. A downside of using integer commitments is that they are parameterized by values that have to be sampled properly. Specifically, integer commitments are parameterized by a tuple (n, g, h) , where n denotes a suitable RSA modulus and g, h are carefully chosen elements generating the same subgroup in $\mathbb{Z}_n^*$. To commit to an integer $m \in \mathbb{Z}$, one computes $c = g^m h^r \bmod n$, where r is a random value chosen from a suitable domain. To decommit, it suffices to reveal m and r . For the binding property to hold, the parameters must not be chosen by the

	Schnorr dlog	Our protocol
Computation (P)	$\ell \cdot \kappa$	1
Computation (V)	$\ell \cdot \kappa$	$1 + \ell$
Communication	$\ell \cdot \kappa \cdot (1 + 2\mathbf{n} + \kappa_s)$	$\ell \cdot \kappa + 2\mathbf{n} + \kappa + \kappa_s$

Table 1: Efficiency comparison between our protocol and the “canonical” Schnorr discrete-log proof with bit challenges. In the table, ℓ denotes the commitment vector length, $\mathbf{n}$ denotes the bitlength of n , and $\kappa \in \{\kappa_s, \kappa_c\}$ is chosen depending on whether the protocol is run interactively (statistical parameter κ_s) or via the Fiat-Shamir transform (computational parameter κ_c) to compute the verifier’s challenges. Computation denotes the number of exponentiations modulo n performed by each party.

committer,¹ while for the hiding property, the parameters must be well formed so that g lies in the multiplicative group generated by h . Unfortunately, no highly efficient protocol is known for proving well-formedness. As a result, when using integer commitments, one must either rely on a trusted setup phase or use a rather inefficient protocol (where communication and computation scale linearly with the computational security parameter).

We introduce, to the best of our knowledge, the first highly efficient parameter well-formedness protocol. The resulting protocol guarantees that the quotient group $\langle g, h \rangle / \langle h \rangle$ is small.² Although it does not guarantee perfect hiding, the resulting hiding property is sufficient for many applications. We also present the natural extension to vector commitments, which allows a committer to publish a single element $c \in \mathbb{Z}_n^*$ while committing to *many* integers $x_1, \dots, x_k$ (by sampling multiple base elements g_i , rather than a single g). Additionally, we show how to implement our highly efficient parameter well-formedness protocol in the vector setting with essentially zero communication overhead compared to the singleton case. table 1 compares the computation and communication costs of our well-formedness protocol to the “canonical” approach.³ To illustrate, in the interactive setting with vectors of size $\ell = 3$ and for statistical parameter $\kappa_s = 40$, the canonical protocol requires roughly $\ell\kappa_s > 120$ times more communication, and the prover’s and verifier’s computation scale by factors of 120 and 40, respectively. Note also that the non-interactive, Fiat–Shamir setting saves $\ell \cdot \kappa_s$ bits of communication for both protocols (compared to the interactive case), so the communication complexity of our protocol essentially boils down to sending two $\mathbf{n}$ -bit values—about 0.5 KB for a 2048-bit RSA modulus—whereas the non-interactive canonical protocol requires about 60 KB.

What we do not cover. This paper does not aim to cover every known aspect of integer commitments or their related protocols and extensions. Rather, this paper is intended as a detailed and self-contained tutorial for using RSA-based integer commitments and to showcase a novel parameter well-formedness proof. In particular, we do not discuss commitments based on non-RSA assumptions (e.g., class groups [Cou+21]) or formally present the four-square decomposition

¹If the committer knows the factorization of n or a discrete-log relation between g and h , the binding property can be easily broken.

²Intuitively, the smaller the quotient group, the smaller the order of the “cheating element” σ is for $g = \sigma h'$ and $h' \in \langle h \rangle$.

³For completeness, we recall that the canonical approach consists of proving that $\langle g, h \rangle / \langle h \rangle = \{1\}$ (equivalently, $g \in \langle h \rangle$) via the standard Schnorr proof of knowledge of the discrete logarithm of g with respect to h , where the verifier’s challenge consists of bits (rather than larger values). To achieve the desired soundness, this proof must be repeated κ_s times in the interactive setting (or κ_c times in the computational setting).

technique [Lip03] for *tight* range proofs. The reader is referred to [Chr+24] for a more detailed survey on range proofs.

1.2 Related work on Range Proofs

Proofs with slackness. As is also the case for the range proof presented in this paper, the range proofs [FO97; Bou00; DF02] presented herein exhibit some *slackness*: when proving that a committed value m lies in some range $[-a, a]$, it is possible for a cheating prover to make the verifier accept witnesses from a slightly larger set $[-a \cdot 2^\varepsilon, a \cdot 2^\varepsilon]$ where ε is some value known to all. This slackness does not affect many applications of interest, e.g., threshold ECDSA [Lin17; LN18; GG18; Can+20] (and removing it incurs substantial overhead, as discussed below), so we treat it as a quirk of the proofs.

The four-square decomposition technique. Starting with Lipmaa [Lip03] (and later refined by Groth [Gro05] and Couteau et al. [CPP17]), the four-square decomposition technique for proving $m \in [a, b]$ involves showing that both $m - a$ and $-m + b$ are nonnegative. Specifically, by Lagrange’s theorem, any nonnegative integer can be written as a sum of four squares, and there are known efficient algorithms for finding this decomposition. The last step involves proving, for example, that $m - a = a_0^2 + a_1^2 + a_2^2 + a_3^2$ by committing to each a_i separately and using Schnorr-style protocols. However, the overhead of finding the square decomposition and making these additional commitments can be quite costly.

The bit-decomposition technique. Alternatively, one can avoid integer commitments entirely by committing to the bit (or u -ary) decomposition of m , i.e., $m = \sum_j m_j 2^j$, and using a bit (or u -ary) commitment scheme [Bri+88; CCs08; Gro11]. General zero-knowledge techniques—such as bulletproofs [Bün+18]—are then used to show that the committed integer lies within the desired range. However, this approach inherently results in super-constant proof sizes (which scale linearly with the witness size) or incurs the overhead of using general-purpose SNARKs like Groth [Gro16].

On [Cou+21] and SHARP [Cou+22]. In conclusion, we mention the work of Couteau et al. [Cou+21] and its follow-up [Cou+22], which introduce so-called bounded integer commitments that behave like integer commitments but operate on rationals rather than integers. They can be instantiated in various groups (including known-order groups) in combination with the square-decomposition technique to obtain tight range proofs. In [Cou+22], the authors show that these proofs achieve performance comparable to the bulletproofs approach.

Paper Organization

A rather detailed preliminaries section is provided in Section 2. The integer commitment scheme is described and analyzed in Section 3, and the strong-RSA-based range proof is described and analyzed in Section 4. In Section 5, we use the range proof protocol for constructing equality-proof protocols for many additively homomorphic commitment schemes of interest. We also present some applications of these proofs. Our new lightweight setup protocol and its security guarantees are given in Section 6. In Section 7, we discuss *vector commitments* and how to generalize the setup protocol with zero communication overhead.

2 Preliminaries

2.1 Notation

We use calligraphic letters to denote sets, uppercase for random variables, and lowercase for integers and functions. We use $\leftarrow$ for assignment, and reserve $=$ for equality. All logarithms are base 2. Let $\mathbb{N}$ denote the set of natural numbers. For $n \in \mathbb{N}$, let $[n] := \{1, \dots, n\}$ and $(n) := \{0, \dots, n\}$. For $n \in \mathbb{N}$, let $\mathbb{Z}_n = \mathbb{Z}/n\mathbb{Z}$ denote the ring of integers modulo n . For group $\mathbb{G}$ and $A \in \mathbb{G}$, let $\langle A \rangle$ be the subgroup generated by A . Specifically, for $z \in \mathbb{Z}$ let $\langle z \rangle_n$ be the subgroup of $\mathbb{Z}_n$ generated by z (with respect to multiplication, i.e., iterated powers), and let $\text{ord}_n(z) := |\langle z \rangle_n|$. For an NP-relation $\mathcal{R}$, let $\mathcal{L}_{\mathcal{R}}$ be the language $\{\phi : \exists w \text{ s.t. } (\phi, w) \in \mathcal{R}\}$.

We use PPT to denote *probabilistic polynomial time* and PPTM to denote PPT machines, i.e., algorithms. A function $f(n)$ is *negligible* if for every $p \in \text{poly}$ there exists $n_0 \in \mathbb{N}$ such that $f(n) < 1/p(n)$ for all $n > n_0$. We write $f(n) \leq \text{neg}(n)$ to denote that $f(n)$ is negligible.

All distributions considered are finite. For a set $\mathcal{X}$, let $\sim \mathcal{X}$ denote the uniform distribution over $\mathcal{X}$, and let $x \stackrel{\text{R}}{\leftarrow} \mathcal{X}$ stand for $x \stackrel{\text{R}}{\leftarrow} \sim \mathcal{X}$, i.e., the process of sampling x uniformly over $\mathcal{X}$. The support of a distribution X over a discrete set $\mathcal{X}$, is defined by $\text{Supp}(X) := \{x \in \mathcal{X} : X(x) > 0\}$. For two distributions/random variables A, B , let $\text{SD}(A, B)$ denote their *statistical distance*. We let $\text{neg}(\kappa)$ denote an arbitrary function in $\kappa_{\mathcal{C}}$.

Security parameters. Throughout, we use two security parameters: $\kappa_{\mathcal{S}}$ used for quantities (functions of the security parameter) that are robust to the adversary running time, e.g., statistical distance, and $\kappa_{\mathcal{C}}$ used for quantities that are sensitive to the adversary running time, e.g., computational distance. In practice, this distinction allows the protocols to use much smaller value for $\kappa_{\mathcal{S}}$ (e.g., 64) than for $\kappa_{\mathcal{C}}$, e.g., 128, leading to more efficient protocols.

To avoid notational clutter, when defining languages and promise problems, defined further below, we consider a single global security parameter κ . (Of course, all these definitions can be parameterized by different values of the security parameter.)

2.2 Sigma Protocols

The zero-knowledge protocols presented in this paper are variants of *Sigma protocols*.⁴ We start with the basic definition due to Damgård [Dam10].

For a protocol $\Pi = (\text{P}, \text{V})$, let $\text{out}(\text{P}(w), \text{V})(\phi)$ denote the parties' outputs in a random execution of protocol Π on common input ϕ and P 's private input w , and let $\text{trans}(\text{P}(w), \text{V})(\phi)$ denote the transcript of the execution, e.g., common input and messages; when clear from the context, we typically simply write $(\text{P}(w), \text{V})(\phi)$ instead of $\text{out}(\text{P}(w), \text{V})(\phi)$.

Definition 2.1 (Sigma protocols). *A Sigma protocol $\Pi = (\text{P}, \text{V})$ for an NP relation $\mathcal{R}$, is a poly-time, three-message, public-coin⁵ protocol with the following properties:*

Perfect correctness. *For any $(\phi, w) \in \mathcal{R}$: $\Pr[\text{out}(\text{P}(w), \text{V})(\phi) = \text{true}] = 1$.*

⁴We use the sometimes non-standard Sigma protocol formulations given below, since it is more informative than just stating the security of the different protocols are proof-of-knowledge, honest-verifier statistical zero knowledge.

⁵The verifier sends uniformly chosen messages.

Honest-verifier statistical zero knowledge. *There exists a PPTM Sim such that for any $(\phi, w) \in \mathcal{R}$:*

$$\text{SD}(\text{trans}((P(w), V)(\phi)), \text{Sim}(\phi)) \leq 2^{-|\phi|}.$$

Special soundness. *There exist PPTM Ext such that for any $\hat{P}$:*

$$\Pr_{(\phi, \alpha, \beta_0, \beta_1, \gamma_0, \gamma_1) \leftarrow \hat{P}} \left[\begin{array}{c} \forall j \in \{0, 1\}: V(t_j \leftarrow (\phi, \alpha, \beta_j, \gamma_j)) = \text{true} \\ (\phi, \text{Ext}(t_0, t_1)) \notin \mathcal{R} \end{array} \right] \leq \text{neg}(|\phi|) \quad (1)$$

Namely, given two accepting transcripts for ϕ with the same first message, the extractor almost always manages to output a witness for ϕ . It follows that even an unbound cheating prover, cannot make the verifier accept $x \notin \mathcal{L}(\mathcal{R})$.

Extended Sigma protocols. *An extended Sigma protocol has a preamble stage that is independent of the parties' input. The special soundness extractor is given the prover's view in this stage.*

Parameterized protocols. If the protocol is parameterized by some value $\rho \in \{0, 1\}^*$, for the above we view it as part of the common input, e.g., the statistical distance should be bounded by $\text{neg}(|\phi, \rho|)$.

Computational variant. The special soundness property of Sigma protocols is naturally relaxed to the computational settings, referred to as **computational special soundness**, in which it is only required to hold for efficiently generated transcripts.

Definition 2.2 (Computational Sigma protocols). *A computational Sigma protocol (P, V) for $\mathcal{R}$ is a Sigma protocol for $\mathcal{R}$ but Equation (1) should only holds for any PPTM $\hat{P}$. If (P, V) is an extended Sigma protocol, then $\hat{P}$ and V interact in the preamble stage before $\hat{P}$ outputs $(\phi, \alpha, \beta_0, \beta_1, \gamma_0, \gamma_1)$.*

Computational special soundness immediately implies the so-called *proof of knowledge extractor*: after the first message is sent, use rewinding to generate two accepting transcripts and apply the computational special soundness extractor.

Distributional variant. In some cases we do not know how to construct a (information theoretic or computational) Sigma protocol that is sound for *any* instance. Rather, we relax the special soundness property to hold only for “typical” instances. For concreteness, we only focus on the computational variant.

Definition 2.3 (Computational Sigma protocols with respect to distributions). *A three-message, public-coin protocol (P, V) is a computational Sigma protocol with respect to a family of relations $\{\mathcal{R}_p\}_p$ and distribution ensemble $\{P_\kappa\}_\kappa$, if the following hold:*

Correctness. *For any $\kappa \in \mathbb{N}$ and $p \in \text{Supp}(P_\kappa)$: (P_p, V_p) , i.e., p is given as an additional parameter to the parties, is perfectly correct protocol for $\mathcal{R}_p$.*

Zero knowledge. *There exists PPTM Sim such that for any $\kappa \in \mathbb{N}$ and $p \in \text{Supp}(P_\kappa)$: (P_p, V_p) is honest-verifier statistical zero-knowledge protocol for $\mathcal{R}_p$, and it is realized by Sim_p .*

Computational special soundness. *There exists PPTM Ext such that for any $\kappa \in \mathbb{N}$, with all but negligible probability over $p \xleftarrow{R} P_\kappa$: (P_p, V_p) has computational special soundness, and it is realized by Ext_p .*

Promise problems. We consider Sigma protocols for *promise problems*. A promise problem is a union of two distinct sets, the “Yes instances” and the “No instances”. Throughout, we define a promise problem $\Pi = \Pi^{\text{Yes}} \cup \Pi^{\text{No}}$ by defining Π^{Yes} and a “Slack subset” $\Pi^{\text{Slack}} \supseteq \Pi^{\text{Yes}}$. The No instances Π^{No} are then implicitly defined by $\neg \Pi^{\text{Slack}}$. For instance, the Yes instance can be $\Pi^{\text{Yes}} = \{(A, a) : A = a \cdot G : a \in [100]\}$, where G is an (additive) group generator, and the Slack instances can be $\Pi^{\text{Slack}} = \{(A, a) : A = a \cdot G : a \in [1000]\}$.

Sigma protocols are naturally generalized to promise problems as follows.

Definition 2.4 (Sigma protocols for promise problems). A Sigma protocol for a promise problem $\Pi = \Pi^{\text{Yes}} \cup \Pi^{\text{No}}$ is a three-message, public-coin, where

1. *Completeness and honest-verifier zero knowledge only guaranteed to hold for $x \in \Pi^{\text{Yes}}$.*
2. *The knowledge extractor is guaranteed to output w with $(c, w) \notin \Pi^{\text{No}}$.*

Computational Sigma protocols for promise problems, also with respect to distributions, are naturally defined.

For instance, given a Sigma protocol for the promise problem Π we considered above, we are guaranteed that the extractor outputs a “not too large” discrete log of A . While this is a weaker security guarantee, this slackness sometimes allows simpler/more efficient protocols. A second advantage of using promise problems is a not handling of bad “parameters”, e.g., weak public keys. See Sections 4 and 5.1.3.

2.3 Quadratic Residuosity for Primes and Biprimes

2.3.1 Basic Notation

Let $\mathcal{P}$ be the set of all primes. A prime number $p \in \mathcal{P}$ is *safe* if $p = 2p' + 1$ for some prime $p' > 2$. (Note that if p is a safe prime then $p = 3 \bmod 4$, since if $p = 1 \bmod 4$ then $p = 4k + 1$ and so $p' = 2k$ which is impossible). A biprime, i.e., a product of two primes, is *safe*, or a *strong RSA modulus*, if it is the product of two distinct safe primes.

Let $(x)_p := x \bmod p$. Note that $(x \cdot y)_p = (x)_p \cdot (y)_p \bmod p$. For a biprime $n = pq$, we use the “CRT function” $\text{crt} : \mathbb{Z}_n \mapsto \mathbb{Z}_p \times \mathbb{Z}_q$ defined by $\text{crt}(x) = ((x)_p, (x)_q)$. It is well known that crt is a bijection from $\mathbb{Z}_n^*$ to $\mathbb{Z}_p^* \times \mathbb{Z}_q^*$ (and also from $\mathbb{Z}_n$ to $\mathbb{Z}_p \times \mathbb{Z}_q$). A value $x \in \mathbb{N}$ is a quadratic residue modulo p if there exists a $y \in \mathbb{N}$ such that $x = y^2 \bmod p$. We denote the set of quadratic residues modulo p by $\mathcal{QR}_p$. That is, $\mathcal{QR}_n := \{y \in \mathbb{Z}_n^* : \exists x \in \mathbb{Z}_n^* \text{ s.t. } x^2 = y \bmod n\}$. We denote $-\mathcal{QR}_n = \{-x : x \in \mathcal{QR}_n\}$ and $\pm \mathcal{QR}_n = \mathcal{QR}_n \cup -\mathcal{QR}_n$.

2.3.2 Jacobi Symbol

The Jacobi symbol for primes and biprimes is defined as follows.

Definition 2.5 (Jacobi symbol). *The Jacobi symbol of x modulo $p \in \mathbb{P}$ is defined by*

$$\mathcal{J}_p(x) := \begin{cases} +1 & \text{if } x \text{ is a quadratic residue modulo } p \\ -1 & \text{if } x \text{ is not a quadratic residue modulo } p. \end{cases}$$

The Jacobi symbol modulo a biprime $n = p \cdot q$, is defined by $\mathcal{J}_n(x) = \mathcal{J}_p(x) \cdot \mathcal{J}_q(x)$.

It is well-known that for any prime p : $\mathcal{J}_p(x) = 1$ iff $x \in \mathcal{QR}_p$. But modulo a composite, the Jacobi symbol is *not* an indication of quadratic residuosity: it is true that every quadratic residue has Jacobi symbol 1, but there are also an equal number of non-quadratic residues with Jacobi symbol 1. Yet, the following facts hold:

Proposition 2.6 (Properties of the Jacobi symbol for biprimes). *For any biprime $n = p \cdot q$ with $p, q > 2$:*

1. $\mathcal{J}_n(x \cdot y) = \mathcal{J}_n(x) \cdot \mathcal{J}_n(y)$.
2. $\Pr_{x \leftarrow \mathbb{Z}_p^*}[\mathcal{J}_p(x) = 1] = 1/2$.
3. *If $p = q = 3 \bmod 4$, then for any $x \in \mathbb{N}$:*
 - (a) $\mathcal{J}_p(x) = -\mathcal{J}_p(-x)$ and $\mathcal{J}_q(x) = -\mathcal{J}_q(-x)$.
 - (b) $\mathcal{J}_n(x) = \mathcal{J}_n(-x)$.
4. *If $p = q = 3 \bmod 4$, then for $u, v \in \mathbb{Z}_n^*$ with $u^2 = v^2 \bmod n$ and $\mathcal{J}_n(u) \neq \mathcal{J}_n(v)$, it holds that $\gcd((u + v) \cdot (u - v), n) > 1$.*

Proof.

1. Since $\mathcal{J}_p(x) = x^{\frac{p-1}{2}}$ it follows that $\mathcal{J}_p(x \cdot y) = \mathcal{J}_p(x) \cdot \mathcal{J}_p(y)$. By definition, $\mathcal{J}_n(x \cdot y) = \mathcal{J}_p(x \cdot y) \cdot \mathcal{J}_q(x \cdot y)$. Thus, $\mathcal{J}_n(x \cdot y) = \mathcal{J}_p(x) \cdot \mathcal{J}_p(y) \cdot \mathcal{J}_q(x) \cdot \mathcal{J}_q(y) = (\mathcal{J}_p(x) \cdot \mathcal{J}_q(x)) \cdot (\mathcal{J}_p(y) \cdot \mathcal{J}_q(y)) = \mathcal{J}_n(x) \cdot \mathcal{J}_n(y)$.
2. This holds if half of the elements of $\mathbb{Z}_p^*$ are quadratic residues. Since $\mathbb{Z}_p^*$ is a cyclic group, it has a generator g . Thus, $1, g^2, g^4, \dots, g^{p-1}$ are all quadratic residues, implying that there are at least $\frac{p-1}{2}$ quadratic residues. Define the polynomial $f(x) = x^{\frac{p-1}{2}} - 1 \bmod p$. Each root of the polynomial f is a quadratic residue (since a root x is such that $x^{\frac{p-1}{2}} = 1 \bmod p$ and by Proposition 2.7 this implies that it is a quadratic residue). Now, a degree $\frac{p-1}{2}$ polynomial has at most $\frac{p-1}{2}$ roots. We conclude that there are *exactly* $\frac{p-1}{2}$ quadratic residues, implying that exactly half the elements of $\mathbb{Z}_p^*$ are quadratic residues.
3.
 - (a) Since $p = 3 \bmod 4$, there exists a value $k \in \mathbb{N}$ such that $p = 4k + 3$. Thus $(-1)^{\frac{p-1}{2}} = (-1)^{2k+1} = -1$. By Proposition 2.7, $\mathcal{J}_p(-x) = (-x)^{\frac{p-1}{2}} = (-1)^{\frac{p-1}{2}} \cdot x^{\frac{p-1}{2}} = -\left(x^{\frac{p-1}{2}}\right) = -\mathcal{J}_p(x)$.
 - (b) Compute $\mathcal{J}_n(x) = \mathcal{J}_p(x) \cdot \mathcal{J}_q(x) = \mathcal{J}_p(-x) \cdot \mathcal{J}_q(-x) = \mathcal{J}_n(-x)$.

4. $u^2 - v^2 = (u + v) \cdot (u - v) = 0 \pmod n$. By item 3, $u \neq -v$ (since if $u = -v \pmod n$ then it would follow that $\mathcal{J}_n(u) = \mathcal{J}_n(v)$). Now, $n \mid (u + v) \cdot (u - v)$ and thus $p \mid (u + v) \cdot (u - v)$. Since p is prime, $p \mid (u + v)$ or $p \mid (u - v)$; assume $p \mid (u + v)$; the proof for the other case is similar. If $q \mid (u + v)$ as well, then $n \mid (u + v)$, but this implies that $u + v = 0 \pmod n$ and so $u = -v \pmod n$. Thus, $p \mid (u + v)$ and $q \nmid (u + v)$ implying that $\gcd(n, u + v) = p > 1$. $\square$

The Jacobi symbol can be easily computed.

Proposition 2.7 (Computing the Jacobi symbol). *Let $p > 2$ be a prime. Then, $\mathcal{J}_p(x) = x^{\frac{p-1}{2}} \pmod p$.*

Proof. Since p is a prime, it follows that $\mathbb{Z}_p^*$ is a cyclic group; let g be a generator of the group. If x is a quadratic residue, then $x = g^i$ for some even $i = 2j$. Using the fact that $z^{p-1} \pmod p = 1$ for every z , we have

$$x^{\frac{p-1}{2}} = (g^{2j})^{\frac{p-1}{2}} = g^{(p-1) \cdot j} = 1^j = 1 \pmod p.$$

If x is not a quadratic residue, then $x = g^i$ for some odd $i = 2j + 1$. Thus,

$$x^{\frac{p-1}{2}} = (g^{2j+1})^{\frac{p-1}{2}} = (g^{2j})^{\frac{p-1}{2}} \cdot g^{\frac{p-1}{2}} = g^{\frac{p-1}{2}} \pmod p.$$

Since $(g^{\frac{p-1}{2}})^2 = g^{p-1} = 1 \pmod p$ we have that $g^{\frac{p-1}{2}}$ is a square root of 1. The only square roots of 1 are 1, -1. Now, since g is a generator, it follows that $g^{\frac{p-1}{2}} \neq 1 \pmod p$ and thus $x^{\frac{p-1}{2}} = g^{\frac{p-1}{2}} = -1$. $\square$

2.3.3 The Group $\mathcal{QR}_n$

It is easy to verify that $\mathcal{QR}_n$ is a multiplicative group, since the product of two quadratic residues is a quadratic residue. We make use of the following observation that an element x of $\mathbb{Z}_n^*$ is a quadratic residue relative to n if and only if x is a quadratic residue relative to both p and q .

Proposition 2.8 (Structure of $\mathcal{QR}_n$). *For any biprime $n = p \cdot q$ and $x \in \mathbb{Z}_n^*$: $x \in \mathcal{QR}_n$ iff $\text{crt}(x) \in \mathcal{QR}_p \times \mathcal{QR}_q$.*

Proof. Let $x_p = x \pmod p$ and $x_q = x \pmod q$.

Assume that $x \in \mathcal{QR}_n$. Let $y \in \mathbb{Z}_n^*$ be such that $x = y^2 \pmod n$. Since $n \mid x - y^2$ it holds that both $p \mid x - y^2$ and $q \mid x - y^2$ and $x = y^2 \pmod p$ and $x = y^2 \pmod q$, implying that $x_p = y_p^2 \pmod p$ and $x_q = y_q^2 \pmod q$.

Assume $x_p \in \mathcal{QR}_p$ and $x_q \in \mathcal{QR}_q$. Let y_p and y_q be such that $x = y_p^2 \pmod p$ and $x = y_q^2 \pmod q$. By the CRT, there exists a unique $y \in \mathbb{Z}_n$ such that $y = y_p \pmod p$ and $y = y_q \pmod q$. Since $y^2 = x \pmod p$ and $y^2 = x \pmod q$ and since p, q are prime (and so relatively prime), it follows that $y^2 = x \pmod n$, as required. $\square$

Proposition 2.9. *For every $x \in \mathcal{QR}_n$ there are four square roots, two of which have Jacobi symbol 1 and two of which have Jacobi symbol -1.*

Proof. By Proposition 2.8, it holds that if $x \in \mathcal{QR}_n$ then $x_p \in \mathcal{QR}_p$ and $x_q \in \mathcal{QR}_q$, where $x_p = x \bmod p$ and $x_q = x \bmod q$. Let $a, -a$ be the square roots of x in $\mathbb{Z}_p^*$, and let $b, -b$ be the square roots of x in $\mathbb{Z}_q^*$. By the Chinese remainder theorem, the combinations $(a, b), (-a, -b), (a, -b), (-a, b)$ map to 4 different elements in $\mathbb{Z}_n$; denote them y_1, y_2, y_3, y_4 . Furthermore, by Proposition 2.8, each element is a square root and it holds that $(a, b) = \text{crt}(y_1)$, $(-a, -b) = \text{crt}(y_2)$, $(a, -b) = \text{crt}(y_3)$, and $(-a, b) = \text{crt}(y_4)$. Clearly, $y_2 = -y_1 \bmod n$ and $y_4 = -y_3 \bmod n$ since if $a = y_1 \bmod p$ then $-a = -y_1 \bmod p$, and likewise for all other values. By Proposition 2.6(3), $\mathcal{J}_p(a) = -\mathcal{J}_p(-a)$ and $\mathcal{J}_q(b) = -\mathcal{J}_q(-b)$, and by definition $\mathcal{J}_n(c) = \mathcal{J}_p(c) \cdot \mathcal{J}_q(c)$. We therefore conclude that if $\mathcal{J}_p(a) = \mathcal{J}_q(b) = 1$, then $\mathcal{J}_n(y_1) = \mathcal{J}_n(y_2) = 1$ and $\mathcal{J}_n(y_3) = \mathcal{J}_n(y_4) = -1$; likewise, for all combinations of values of $\mathcal{J}_p(a)$ and $\mathcal{J}_q(b)$. $\square$

Proposition 2.10. $-1 \notin \mathcal{QR}_n$ for any biprime $n = p \cdot q$ with $p = q = 3 \bmod 4$.

Proof. Since $p = 3 \bmod 4$, $\mathcal{J}_p(-1) = -1 \bmod p$ and therefore $-1 \notin \mathcal{QR}_p$. Thus, the proof follows by Proposition 2.8. $\square$

Proposition 2.11. For every $y \in \mathbb{Z}_n^*$, $\mathcal{J}_n(y) = 1$ if and only if $y \in \mathcal{QR}_n$ or $-y \in \mathcal{QR}_n$.

Proof. If $y \in \mathcal{QR}_n$ then $\mathcal{J}_n(y) = 1$ (by Proposition 2.8). Furthermore, by Proposition 2.6(3), we have that $\mathcal{J}_n(y) = \mathcal{J}_n(-y)$. Thus, it also holds that if $-y \in \mathcal{QR}_n$ then $\mathcal{J}_n(y) = 1$. For the other direction, if $\mathcal{J}_n(y) = 1$ then either $\mathcal{J}_p(y) = \mathcal{J}_q(y) = 1$ or $\mathcal{J}_p(y) = \mathcal{J}_q(y) = -1$ (since $\mathcal{J}_n(y) = \mathcal{J}_p(y) \cdot \mathcal{J}_q(y)$). In the former case, this implies that $y \in \mathcal{QR}_n$ (by Proposition 2.8). In the latter case, observe that $\mathcal{J}_p(-y) = \mathcal{J}_q(-y) = 1$ (since $\mathcal{J}_p(-1) = \mathcal{J}_q(-1) = -1$) and so $-y \in \mathcal{QR}_n$. $\square$

2.3.4 Structure of $\mathbb{Z}_n^*$

Proposition 2.12 (Structure of $\mathbb{Z}_n^*$). For any safe biprime $n = p \cdot q$ with $p = 2p' + 1$ and $q = 2q' + 1$, it holds that $\mathbb{Z}_n^*$ is isomorphic to $\mathbb{Z}_2 \times \pm \mathcal{QR}_n$. Specifically, for any $\alpha \in \mathbb{Z}_n^* \setminus \pm \mathcal{QR}_n$, it holds that $\phi(b, r) = \alpha^b \cdot r \bmod n$ is an isomorphism from $\mathbb{Z}_2 \times \pm \mathcal{QR}_n$ to $\mathbb{Z}_n^*$.

Proof. First we prove that ϕ is a bijection. Fix $\alpha \in \mathbb{Z}_n^* \setminus \pm \mathcal{QR}_n$, and consider the inverse mapping

$$\phi^{-1}(x) := \begin{cases} (0, x) & x \in \pm \mathcal{QR}_n \\ (1, \alpha^{-1} \cdot x \bmod n) & \text{otherwise} \end{cases}.$$

It is clear that $\phi(\phi^{-1}(x)) = x$ for any $x \in \mathbb{Z}_n^*$: for $x \in \pm \mathcal{QR}_n$, $\phi^{-1}(x) = (0, x)$ and $\phi(0, x) = \alpha^0 \cdot x = x \bmod n$; for $x \notin \pm \mathcal{QR}_n$, $\phi^{-1}(x) = (1, \alpha^{-1} \cdot x)$ and $\phi(1, \alpha^{-1} \cdot x) = \alpha^1 \cdot \alpha^{-1} \cdot x = x \bmod n$. Thus, ϕ^{-1} is a 1-1 function.

We now prove that it maps all of $\mathbb{Z}_n^*$ onto $\mathbb{Z}_2 \times \pm \mathcal{QR}_n$. First, for any $x \in \pm \mathcal{QR}_n$ we have that $\phi^{-1}(x) = (0, x)$. Thus, clearly $\{\phi^{-1}(x) \mid x \in \pm \mathcal{QR}_n\} = \{0\} \times \pm \mathcal{QR}_n$. Next, for $x \notin \pm \mathcal{QR}_n$ we have that $\phi^{-1}(x) = (1, \alpha^{-1} \cdot x)$. Thus, it suffices to show that $\{\alpha^{-1} \cdot x \mid x \in \pm \mathcal{QR}_n\} = \pm \mathcal{QR}_n$, since that implies that $\{\phi^{-1}(x) \mid x \notin \pm \mathcal{QR}_n\} = \{1\} \times \pm \mathcal{QR}_n$. Defining $f(x) = \alpha^{-1} \cdot x \bmod n$, it remains to show that f is a bijection over $\pm \mathcal{QR}_n$.

The fact that f is invertible is immediate. We therefore just need to show that for any $\alpha, x \in \mathbb{Z}_n^* \setminus \pm \mathcal{QR}_n$ it holds that $\alpha^{-1} \cdot x \in \pm \mathcal{QR}_n$. By Proposition 2.11, $\mathcal{J}_n(\alpha) = \mathcal{J}_n(x) = -1$ (since neither are in $\pm \mathcal{QR}_n$). Furthermore, $\mathcal{J}_n(\alpha^{-1}) = \mathcal{J}_n(\alpha)$ (since $\alpha \cdot \alpha^{-1} = 1$ and $\mathcal{J}_n(1) = 1$). Thus,

$$\mathcal{J}_n(\alpha^{-1} \cdot x) = \mathcal{J}_n(\alpha^{-1}) \cdot \mathcal{J}_n(x) = -1 \cdot -1 = 1$$

and so $\alpha^{-1} \cdot x \in \pm \mathcal{QR}_n$, as required. $\square$

2.4 The Strong RSA Assumption

We use the following algorithm for sampling RSA moduli:

Algorithm 2.13 (StModuliGen). Input: 1^κ , $s \in \mathbb{N}$. The default value of the parameter s , if not set, is 0.

Operation:

1. Sample uniformly two distinct $\max\{\kappa_c, \lfloor \log(s)/2 + 1 \rfloor\}$ -bit safe-primes p, q .
2. Return $n \leftarrow p \cdot q$

That is, StModuliGen returns a strong RSA modulus n with the additional property that $n \geq s$.

The strong RSA assumption states that given a modulus n generated by StModuliGen and a random element $y \in \mathbb{Z}_n^*$, it is hard to find a pair (e, x) such that $x^e = y \bmod n$ for $e \notin \{1, -1\}$. The difference between this and the standard RSA assumption is that e is not fixed and can be chosen by the adversary.

Assumption 2.14 (Strong RSA). *For any PPTM A :*

$$\Pr_{n \leftarrow \text{StModuliGen}(1^\kappa); y \leftarrow \mathbb{Z}_n^*} \left[(e, x) \xleftarrow{R} A(1^\kappa, n, y) : e \in \mathbb{Z}_n^* \setminus \{-1, 1\} \wedge x^e = y \bmod n \right] \leq \text{neg}(\kappa).$$

2.4.1 Two Hard Games

In this section, we consider two games whose hardness follows from strong RSA (assumption 2.14). Recall that an integer commitment to x using randomness r and parameters $n, g, h \in \mathbb{N}$ has the form $g^x \cdot h^r \bmod n$. (See details in Section 3.)

Generating non-trivial commitments. In this game, the player is given properly chosen parameters (n, g, h) , and his goal is to find c, α, β, γ such that $c^\gamma = g^\alpha \cdot h^\beta \bmod n$, and γ divides neither α nor β . Clearly, A can find c, α, β where $c = g^\alpha \cdot h^\beta \bmod n$ by just choosing α, β and computing them (this is just computing an integer commitment). Furthermore, it can easily choose α', β' and γ and can compute $\alpha \leftarrow \alpha' \cdot \gamma$ and $\beta \leftarrow \beta' \cdot \gamma$. Then, by setting $c \leftarrow g^{\alpha'} \cdot h^{\beta'}$, it holds that $c^\gamma = g^\alpha \cdot h^\beta$. In this case, however, γ divides α and β . Thus, the hard problem is finding these values where γ does not divide α and β . The following proposition, stating the hardness of this game, was proven in [FO97; MR01; CMP20].⁶

Proposition 2.15 (Hardness of generating non-trivial commitments). *Assume assumption 2.14 holds and let $s: \mathbb{N} \mapsto \mathbb{N}$ be a non-decreasing function. Then for any PPTM A :*

$$\Pr_{\substack{n \leftarrow \text{StModuliGen}(1^\kappa), h \leftarrow \mathcal{QR}_n \\ \ell \leftarrow [s(n)], g \leftarrow h^\ell \bmod n}} \left[\begin{array}{c} (c, \alpha, \beta, \gamma) \xleftarrow{R} A(1^\kappa, n, g, h) : \\ (c \in \mathbb{Z}_n^*) \wedge (g^\alpha \cdot h^\beta = c^\gamma \bmod n) \wedge (\gamma \nmid \alpha \vee \gamma \nmid \beta) \end{array} \right] \leq \text{neg}(\kappa).$$

Proof. Let A be an algorithm that succeeds in the above with probability $\varepsilon(\kappa)$. We construct an algorithm A' of essentially the same running time that solves the strong RSA problem with

⁶[FO97; MR01; CMP20] require $s(n) \geq n \cdot 2^\kappa$, whereas below we only require $s(n) \geq n$.

probability at least $\varepsilon(\kappa)/10 - \text{neg}(\kappa)$. Recall that algorithm A' receives $(1^\kappa, n, y)$. We assume that $y \in \mathcal{QR}_n$ which happens with probability $1/4$ (by Proposition 2.8 together with the fact that each of $\mathcal{QR}_p$ and $\mathcal{QR}_q$ are half the size of $\mathbb{Z}_p^*$ and $\mathbb{Z}_q^*$ as shown in Proposition 2.6(2)). Note that in this case y is uniform in $\mathcal{QR}_n$. Now, A' sets $h = y$, samples $g \leftarrow h^\ell$ for $\ell \xleftarrow{R} [s]$ and invokes $A(1^\kappa, n, g, h)$ to obtain its output $(c, \alpha, \beta, \gamma)$. If $c \notin \mathbb{Z}_n^*$ or $c^\gamma \neq g^\alpha \cdot h^\beta \pmod n$ or $(\gamma \mid \alpha \text{ and } \gamma \mid \beta)$, then A' halts. Else, A has “succeeded” in its task and A' proceeds.

Since $g = h^\ell \pmod n$ and $c^\gamma = g^\alpha \cdot h^\beta \pmod n$, it holds that $c^\gamma = h^{\ell \cdot \alpha + \beta} \pmod n$. Let $t = \ell \cdot \alpha + \beta$ (note that A' can compute t). Let ℓ^* be the smallest value in $[n]$ such that $h^{\ell^*} = g \pmod n$. Note that given (h, g) , the value ℓ chosen by A' when generating g is uniformly distributed in $\mathcal{L} \leftarrow \{\ell^*, \ell^* + |\langle h \rangle|, \ell^* + 2|\langle h \rangle|, \dots\}$. By Proposition 2.12, the set $\mathcal{QR}_n$ is $1/4$ of the size of $\mathbb{Z}_n^*$, and $h \in \mathcal{QR}_n$. Thus, $|\langle h \rangle| \leq \frac{n}{4}$. Since $g^{\ell^*} = g^{\ell^* + |\langle h \rangle|} \pmod n$ and ℓ is uniform in $[s] \geq [n]$, we conclude that $|\mathcal{L}| \geq 4$.

If $\gamma \nmid \alpha$, if $\gamma \mid (\ell' \cdot \alpha + \beta)$ for some ℓ' , then $\gamma \nmid ((\ell' + |\langle h \rangle|) \cdot \alpha + \beta)$. Otherwise, γ divides both $\ell' \cdot \alpha + \beta$ and $(\ell' + |\langle h \rangle|) \cdot \alpha + \beta$, and thus it divides $|\langle h \rangle| \cdot \alpha$. We claim that $\gcd(\gamma, |\langle h \rangle|) = 1$ since $h \in \mathcal{QR}_n$ and $|\mathcal{QR}_n| = \frac{|\mathbb{Z}_n^*|}{4} = \frac{\phi(n)}{4} = p' \cdot q'$ and so $|\langle h \rangle|$ divides $p' \cdot q'$ implying that it equals either $1, p', q'$ or $p' \cdot q'$. Now, if $\gcd(\gamma, |\langle h \rangle|) \neq 1$ then $\gcd(\gamma, |\langle h \rangle|) \in \{p', q', p' \cdot q'\}$ and thus given γ it is possible to factor n , as follows. This is not immediate since we do not know $|\langle h \rangle|$ and so cannot compute $\gcd(\gamma, |\langle h \rangle|)$. However, we do know that if $n \mid a \cdot b$ then $\frac{n}{\gcd(n, a)} \mid b$. Therefore, $\frac{\gamma}{\gcd(\alpha, \gamma)} \mid |\langle h \rangle|$. Since $\frac{\gamma}{\gcd(\alpha, \gamma)} > 1$ (because $\gamma \nmid \alpha$), we have that $\frac{\gamma}{\gcd(\alpha, \gamma)} \in \{p', q', p' \cdot q'\}$ and so it is possible to factor, in contradiction with the strong RSA assumption (given p' or q' factoring is immediate, and given $p' \cdot q'$ one can compute $\phi(n) = 4 \cdot p' \cdot q'$ which enables factoring n). Let $\mathcal{T} \leftarrow \{\ell' \cdot \alpha + \beta\}_{\ell' \in \mathcal{L}}$. From the above, $|\mathcal{T}| \geq 4$ and γ can divide at most $\left\lceil \frac{|\mathcal{T}|}{2} \right\rceil$ of the elements in the set, and therefore does not divide at least $\frac{2|\mathcal{T}|}{5}$ of the elements of $\mathcal{T}$ ($2|\mathcal{T}|/5$ happens for example if there are five elements and γ divides the first, third and fifth). Thus, if $\gamma \nmid \alpha$, then $\gamma \nmid t$ with probability at least $\frac{2}{5}$.

If $\gamma \mid \alpha$ but $\gamma \nmid \beta$ (recall that all we know is that $\gamma \nmid \alpha \vee \gamma \nmid \beta$), then we have that $\gamma \mid \ell \cdot \alpha$ and $\gamma \nmid \beta$, implying that $\gamma \nmid t = \ell \cdot \alpha + \beta$. Thus, from here on we assume that $\gamma \nmid t$.

Let $\delta \leftarrow \gcd(\gamma, t)$, $\delta_t \leftarrow t/\delta$, $\delta_\gamma \leftarrow \gamma/\delta$, and let v_t, v_γ be such that $v_t \cdot \delta_t + v_\gamma \cdot \delta_\gamma = 1$ (v_t, v_δ can be found efficiently using the extended Euclid algorithm since δ_t, δ_γ are relatively prime). It follows that

$$(c^{v_t} \cdot h^{v_\gamma})^{\delta_\gamma} = c^{v_t \cdot \gamma / \delta} \cdot h^{v_\gamma \cdot \delta_\gamma} = h^{v_t \cdot \delta_t} \cdot h^{v_\gamma \cdot \delta_\gamma} = h^{v_t \cdot \delta_t + v_\gamma \cdot \delta_\gamma} = h \pmod n,$$

where the second equality holds because $c^{v_t \cdot \gamma / \delta} = (c^\gamma)^{v_t / \delta} = (h^{\ell \cdot \alpha + \beta})^{v_t / \delta} = (h^t)^{v_t / \delta} = h^{v_t \cdot t / \delta} = h^{v_t \cdot \delta_t}$. Thus, A' outputs $y = c^{v_t} \cdot h^{v_\gamma}$ and $e = \delta_\gamma$ as the solution to the strong RSA game (since $y^e = h \pmod n$). If $\delta_\gamma = \pm 1$ then $\gamma = \pm \delta$ and so $\gamma \mid t$, in contradiction with our assumption. Thus, A' succeeds in the strong RSA game with probability $\varepsilon(\kappa)/10 - \text{neg}(\kappa)$.⁷ By the strong RSA assumption, this implies that $\varepsilon(\kappa)$ is a negligible function, as required. $\square$

Range proof game. The following game captures the security of the range proof protocol presented in Section 3. The proof of the following proposition is implicit in [CMP20, Sec 6.1].

⁷This probability comes from the fact that if A succeeds and $y \in \mathcal{QR}_n$ (with probability $\frac{1}{4}$) and $\gamma \nmid t$ (with probability $\frac{2}{5}$) and A doesn't factor n (negligible probability), then A' solves the strong RSA problem.

Proposition 2.16 (Hardness of range proof game). *Let $s \geq n$. If assumption 2.14 holds, then for any PPTM A there exists a negligible function neg such that for every $\kappa \in \mathbb{N}$,*

$$\Pr_{\substack{n \xleftarrow{R} \text{StModuliGen}(1^\kappa), h \xleftarrow{R} \mathcal{QR}_n \\ \ell \xleftarrow{R} [s], g \leftarrow h^\ell \bmod n}} \left[(c, \alpha, \beta, \gamma) \xleftarrow{R} A(1^\kappa, n, g, h) : f_{n,g,h}(c, \alpha, \beta, \gamma) \right] \leq \text{neg}(\kappa)$$

where $f_{n,g,h}(c, \alpha, \beta, \gamma)$ is true if and only if

1. $c \in \mathbb{Z}_n^*$, and
2. $\gamma < 2^{\kappa-1}$, and
3. $g^\alpha \cdot h^\beta = c^\gamma \bmod n$, and
4. $\gamma \nmid \alpha$ or $\gamma \nmid \beta$ or $g^{\alpha/\gamma} \cdot h^{\beta/\gamma} \neq \pm c \bmod n$

Proof. Let $(c, \alpha, \beta, \gamma)$ be the output by A when given input $(1^\kappa, n, g, h)$ such that $f_{n,g,h}(c, \alpha, \beta, \gamma) = 1$. By Proposition 2.15, the probability that conditions (1) and (3) hold and $(\gamma \nmid \alpha \text{ or } \gamma \nmid \beta)$ is at most negligible. Thus, assume that $\gamma \mid \alpha$ and $\gamma \mid \beta$, and let $\alpha' \leftarrow \alpha/\gamma$ and $\beta' \leftarrow \beta/\gamma$. Since by assumption $f_{n,g,h}(c, \alpha, \beta, \gamma) = 1$ it follows that $\gcd(\gamma, \phi(n)) \neq 1$, since otherwise it holds that $\gamma^{-1} \bmod \phi(n)$ is well defined, and so

$$g^{\alpha'} \cdot h^{\beta'} = g^{\alpha \cdot \gamma^{-1}} \cdot g^{\beta \cdot \gamma^{-1}} = c^{\gamma \cdot \gamma^{-1}} = c \bmod n, \quad (2)$$

contradicting condition (4).

Next, recall that $n = p \cdot q = (2p' + 1) \cdot (2q' + 1)$ for some κ -bit primes p', q' , and so $\phi(n) = 4 \cdot p' \cdot q'$. Since p', q' are both primes greater than $2^{\kappa-1}$ and since $\gamma < 2^{\kappa-1}$ by condition (2), we deduce that $\gcd(\gamma, p' \cdot q') = 1$. Therefore, $\gamma^{-1} \bmod p' \cdot q'$ exists and γ is *even* (this holds because $\gcd(\gamma, \phi(n)) \neq 1$ and so it must be that γ has a common factor with 2). By Proposition 2.12, we can write $c = r^b \cdot c' \bmod n$ for any $r \in \mathbb{Z}_n^* \setminus \pm \mathcal{QR}_n$, with $b \in \{0, 1\}$ and $c' \in \pm \mathcal{QR}_n$. In particular, we can take $r \in \mathbb{Z}_n^* \setminus \pm \mathcal{QR}_n$ to be a square root of 1 in $\mathbb{Z}_n^*$ (there are four square roots of 1; two of them in $\pm \mathcal{QR}_n$ and two not in $\pm \mathcal{QR}_n$; see Proposition 2.9). Recall that γ is even, and therefore $r^\gamma = 1 \bmod n$. Thus,

$$g^\alpha \cdot h^\beta = c^\gamma = (r^b)^\gamma \cdot c'^\gamma = (r^\gamma)^b \cdot c'^\gamma = 1 \cdot c'^\gamma = c'^\gamma \bmod n$$

and so similarly to Equation (2), it holds that

$$g^{\alpha'} \cdot h^{\beta'} = c' \bmod n.$$

This implies that $c' \neq \pm c \bmod n$, since otherwise $f_{n,g,h}(c, \alpha, \beta, \gamma) = 0$. Let $r = c' \cdot c^{-1} \bmod n$ and note that $g^{\alpha'} \cdot h^{\beta'} = r \cdot c \bmod n$ where $r^\gamma = 1$. This means that the order of r divides γ . Since $\gamma < 2^{\kappa-1}$, by considering the possible divisors of $\phi(n)$, it follows that the order of r is in $\{1, 2, 4\}$. Since there are no elements of order 4 in $\mathbb{Z}_n^*$, and, by assumption, $r \neq \pm 1$, it means that r is a non-trivial root of 1 which allows factoring. □

3 The Integer Commitment Scheme

In this section, we define the integer commitment scheme and prove its basic properties. In the following let $\text{IntCom}_{n,g,h}(x; \rho) := g^x \cdot h^\rho \bmod n$.

Algorithm 3.1 (Integer commitment). The integer commitment scheme $\text{IntComSc} = (\text{Gen}, \text{Commit}, \text{Verify})$ is defined as follows:

Gen: On input $1^{\kappa_s}, 1^{\kappa_c}$ and $s \in \mathbb{N} \cup \{0\}$ ($s \leftarrow 0$ if not set):

1. $n \xleftarrow{\mathbb{R}} \text{StModuliGen}(1^{\kappa_c}, s)$.
2. Sample $h \xleftarrow{\mathbb{R}} \mathcal{QR}_n$ and $\alpha \xleftarrow{\mathbb{R}} [n \cdot 2^{\kappa_s}]$.
3. Let $g \leftarrow h^\alpha \bmod n$.
4. Output $\text{pp} \leftarrow (\kappa \leftarrow (\kappa_s, \kappa_c), n, g, h)$.

Let ExtdGen be the variant of **Gen** that also outputs α .

Commit: On input $\text{pp} = (\kappa, n, g, h)$ and $m \in \mathbb{Z}$:

1. $\rho \xleftarrow{\mathbb{R}} [n \cdot 2^{\kappa_s}]$.
2. Output $\text{IntCom}_{n,g,h}(m; \rho)$.

Verify: On input $\text{pp} = (\kappa, n, g, h)$, $c \in \mathbb{Z}_n^*$ and $m, \rho \in \mathbb{Z}$: Output true iff $\text{IntCom}_{n,g,h}(m; \rho) = \pm c \bmod n$.

Recall that the parameter s enforces StModuliGen to choose $n > s$. When clear from the context, we typically omit κ from pp .

Remark 3.2 (Negative commitments). Rather than defining the **Verify** procedure to (also) accept “negative commitments”, i.e., $\text{IntCom}_{\text{pp}}(x; \rho) = -c \bmod n$, we could have defined $\text{IntCom}_{n,g,h}(x, \rho, b) := g^x \cdot h^\rho \cdot (-1)^b \bmod n$, where $b \in \{0, 1\}$ is uniformly sampled by the committer, and change **Verify** so that $\text{IntCom}_{\text{pp}}(x; \rho, b) = c \bmod n$. It is easy to see that all claims below hold with respect to the modified definition. This would make working with integer commitments less confusing (e.g., does the homomorphism work with negative commitments?), but at the price of deviating from the standard notation.

3.1 Security

In this section, we prove the basic facts about the above scheme.

Claim 3.3 (Homomorphism). *For any $\text{pp} = (n, g, h)$ with $n, g, h \in \mathbb{N}$ and $x, x', \rho, \rho' \in \mathbb{Z}$: $\text{IntCom}_{\text{pp}}(x; \rho) \cdot \text{IntCom}_{\text{pp}}(x'; \rho') = \text{IntCom}_{\text{pp}}(x + x'; \rho + \rho') \bmod n$.*

Proof. Immediate. □

Claim 3.4 (Correctness). *The scheme IntComSc is perfectly correct.*

Proof. Immediate. □

The hiding property of **IntComSc** holds even if the parameters were not chosen as in **Gen** (only requires $g \in \langle h \rangle$).

Claim 3.5 (Statistically hiding). *For any $\text{pp} = ((\kappa_s, \cdot), n, g, h)$ with $n \in \mathbb{N}$, $h \in \mathbb{Z}_n$ and $g \in \langle h \rangle$, it holds that $\text{Commit}_{\text{pp}}$ is $2^{-\kappa_s}$ hiding.*

Proof. Let α be such that $g = h^\alpha \bmod n$. For any $x, x', \rho \in \mathbb{Z}$:

$$\text{IntCom}_{\text{pp}}(x; \rho) = g^x \cdot h^\rho = h^{\rho+\alpha x} = h^{\rho+\alpha(x-x')+\alpha x'} = g^{x'} \cdot h^{\rho+\alpha(x-x')} = \text{IntCom}_{\text{pp}}(x'; \rho') \bmod n$$

for $\rho' = \rho + \alpha \cdot (x - x') \bmod |\langle h \rangle|$. Since the randomness for the commitment is uniform in $2^{\kappa_s} \cdot n$ and $|\langle h \rangle| < n$, we have that both ρ and ρ' are $2^{-\kappa_s}$ close to uniform in $[\langle h \rangle]$. Thus, the distributions over commitments to x and to x' are $2^{-\kappa_s}$ close. $\square$

The binding property of **IntComSc**, with respect to properly chosen parameters, holds if factoring is hard (over the distribution of biprimes induced by **Gen**). Since the binding requires the public parameters to be sampled properly, we explicitly defined its binding property.

Game 3.6 (Integer commitment binding game).

Parameters: $1^{\kappa_c}, 1^{\kappa_s}, s$.

Operation:

1. Challenger sends $\text{pp} \xleftarrow{\text{R}} \text{Gen}(1^{\kappa_s}, 1^{\kappa_c}, s)$ to the player.
2. Player outputs $x_0, \rho_0, x_1, \rho_1 \in \mathbb{Z}$.

The player wins if $x_0 \neq x_1$ and $\text{IntCom}_{n,g,h}(x_0; \rho_0) = \pm \text{IntCom}_{n,g,h}(x_1; \rho_1) \bmod n$.

Claim 3.7 (Computational binding). *For any $s \in \mathbb{N}$, if factoring is hard with respect to the distribution of biprimes induced by $\text{StModuliGen}(1^{\kappa_c}, s)$, then no efficient **A** wins in the binding game on parameters $(1^{\kappa_c}, 1^{\kappa_s}, s)$, for any $\kappa_s \leq \kappa$ with probability larger than $\text{neg}(\kappa_c)$*

Proof. Let **A** be an algorithm that on parameters $(1^{\kappa_c}, 1^{\kappa_s}, s)$, with $\kappa_s \leq \kappa$, wins the game with probability $\varepsilon(\kappa_c)$. Let **F** be the PPT that given arbitrary n and $m \in \mathbb{N}$ so that $\gcd(\phi(n), m) = \phi(n)$, find a non-trivial factor of n .⁸ Consider the following algorithm **B** for factoring a safe prime RSA modulus:

Algorithm 3.8 (B).

Input: $n \in \mathbb{N}$.

Operation:

1. Sample $w \xleftarrow{\text{R}} \mathbb{Z}_n$ and set $h \leftarrow w^2 \bmod n$.
If $\gcd(n, w) \notin (1, n)$ output it and halt.

⁸The existence of **F** is a standard fact [Mil76; Bon02]. Let m' denote the largest odd factor of m . For a random $a \in \mathbb{Z}_n^*$, the last element before 1 in the sequence $a^{m'}, a^{2m'}, \dots, a^{2^i m'}, \dots$ (and this sequence stabilizes to 1 in at most $\log_2(m)$ steps as $a^m = 1 \bmod n$) is a nontrivial square root of 1 with probability at least $1/2$. Such an element x yields a nontrivial factor of n via $\gcd(x - 1, n)$.

2. Sample $\alpha \xleftarrow{R} [n \cdot 2^{\kappa_s}]$ and set $g \leftarrow h^\alpha \bmod n$.
3. Let $(x_0, \rho_0, x_1, \rho_1) \xleftarrow{R} A(1^{\kappa_s}, 1^{\kappa_c}, s, n, g, h)$. Let $x \leftarrow x_0 - x_1$ and $\rho \leftarrow \rho_0 - \rho_1$.
4. Return $F(n, m)$ for $m \leftarrow \rho - \alpha x$.

We conclude the proof showing that

$$\Pr_{n \xleftarrow{R} \text{StModuliGen}(1^{\kappa_c}, s)} [\gcd(n, B(1^{\kappa_c}, n)) \in (1, n)] \leq \text{neg}(\kappa_c) - 2^{-\kappa_s}$$

Consider a random execution on B on $n \xleftarrow{R} \text{StModuliGen}(1^{\kappa_c}, s)$. Note that the distribution on which B calls A upon in this execution is identical to that in the binding game. In the following we consider an execution of B conditioned on A successfully breaking the commitment: $x \neq 0$ and $g^x = \pm h^\rho \bmod n$. Since, by construction, $g, h \in \mathcal{QR}_n$ and since, by Proposition 2.10, $-1 \notin \mathcal{QR}_n$, it holds that $g^x = h^\rho \bmod n$. Therefore $h^m = 1 \bmod n$, and thus

$$m = 0 \bmod |\langle h \rangle_n| \quad (3)$$

Since B samples α uniformly in $[n \cdot 2^{\kappa_s}]$, it follows that for any given (n, g, h) there are at least 2^{κ_s} possible values of α such that $g = h^\alpha \bmod n$. Since, by assumption, $x \neq 0$, we conclude that

$$\Pr[\rho = \alpha x] \leq 2^{-\kappa_s} \quad (4)$$

Thus, it follows that $m \neq 0$ (over the integers), except with probability $2^{-\kappa_s}$. We assume without loss of generality that $\gcd(h - 1, n) = 1$ (otherwise, we are done), then m is a multiple of $\phi(n)/4^9$ and the call to F factors n .¹⁰

□

3.2 Parameters Generation

The above hiding and binding properties hold under certain guarantees regarding the commitment parameters. These guarantees clearly hold if the parameters are *honestly generated* in a trusted setup (using Gen). In practice, however, such a strong trust assumption may be considered unreasonable. Instead, it suffices for the receiver to generate the parameters (n, g, h) and provide a zero-knowledge proof that $g \in \langle h \rangle$. To understand why this is sufficient, consider the two corruption cases:

Committer is corrupt. In this case, the receiver is assumed to be honest (or else, there is no security required). As a result, the parameters are honestly generated and the proof of binding holds as proven.

Receiver is corrupt. In this case, the security property required is that of hiding (binding is needed for a corrupted committer, but here the committer is honest). Observe that the proof of hiding holds for *all* integers n , and only requires that $h \in \mathbb{Z}_n^*$ and $g \in \langle h \rangle$. As such, if the receiver provides a zero-knowledge proof that $g \in \langle h \rangle$ then as long as the committer verifies that $h \in \mathbb{Z}_n^*$ and the ZK proof that $g \in \langle h \rangle$ then hiding is guaranteed.

⁹If $\gcd(h - 1, n) = 1$, then h generates the entire group $\mathcal{QR}_n$ and $|\mathcal{QR}_n| = \phi(n)/4$

¹⁰See footnote 8.

Unfortunately, there is no highly efficient ZK proof for the verifier to use for proving that $g \in \langle h \rangle$. In Section 6, we address this issue by presenting a highly efficient protocol for proving that g is “not far” from $\langle h \rangle$. While the resulting protocol does not guarantee full hiding, its hiding property suffices for many applications.

4 Range Proof based on Strong RSA

The main reason for using integer commitments is that they have highly efficient *range proof* protocols. In a range proof protocol, the prover convinces the verifier it knows an opening (a, ρ) of a commitment $\hat{A}$ with $a \in [d]$. Ideally, we would like to have a sigma protocol for the relation, quantified by a range parameter $d_v \in \mathbb{N}$ and public key $\text{pp} = (n, g, h) \in \mathbb{N}^3$:

$$\mathcal{R}_{\text{pp}}^{\text{Rng}} := \{(c, (a, \rho)) : \text{IntCom}_{\text{pp}}(a; \rho) = c \wedge a \in [d_v]\}$$

For the sake of this tutorial, we focus on a sigma protocol for a relaxation of the above relation formulated as the following promise problem.

Definition 4.1 (Π^{Rng}). For $s, d_v, d_r \in \mathbb{N}$ and $\text{pp} = (n, g, h) \in \mathbb{N}^3$, let

$$\Pi_{s, d_v, d_r, \text{pp}}^{\text{Rng}} := \begin{cases} \text{Yes} : & \{(c, (a, \rho)) : \text{IntCom}_{\text{pp}}(a; \rho) = c \wedge a \in [d_v], \rho \in [d_r]\} \\ \text{Slack} : & \{(c, (a, \rho)) : \text{IntCom}_{\text{pp}}(a; \rho) = \pm c \wedge a \in [\pm s \cdot d_v]\} \end{cases}$$

That is, the slack instances enable the input to be larger than the designated range $[d_v]$ of the yes instances (in particular, $x \in [\pm s \cdot d_v]$). Unfortunately, we are not aware of a sigma protocol (or any highly efficient zero-knowledge POK protocol) for the above problem. For instance, it is not clear how to extract when auxiliary information about the parameters (e.g., factoring of n) is known to the prover. Yet, the following Schnorr-like proof has soundness against uniformly sampled pp .

Protocol 4.2 (Range proof for integer commitments).

Parties: P, V.

Parameters: $1^{\kappa_s}, 1^{\kappa_c}, d_v, d_r \in \mathbb{N}, \text{pp} = (n, g, h) \in \mathbb{N}^3$.

Common input: $c \in \mathbb{N}$.

P’s private input: $a, \rho \in \mathbb{N}$.

Operation:

1. P:
 - (a) Sample $a' \xleftarrow{\mathbb{R}} [2^{\kappa_s + \kappa_c} \cdot d_v]$ and $\rho' \xleftarrow{\mathbb{R}} [2^{\kappa_s + \kappa_c} \cdot \max\{n, d_r\}]$.
 - (b) Send $c' \leftarrow \text{IntCom}_{\text{pp}}(a'; \rho')$ to V.
2. V: Send $e \xleftarrow{\mathbb{R}} [2^{\kappa_c}]$ to P.
3. P:
 - (a) $a'' \leftarrow e \cdot a + a'$.

- (b) $\rho'' \leftarrow e \cdot \rho + \rho'$.
 - (c) Send (a'', ρ'') to V .
4. V : Verify:
- (a) $c, c' \in \mathbb{Z}_n^*$.^a
 - (b) $a'' \in [2^{\kappa_s + \kappa_c + 1} \cdot d_v]$.
 - (c) $\text{IntCom}_{\text{pp}}(a''; \rho'') = c^e \cdot c' \bmod n$.

^aAssuming factoring n is hard, in a setting where the factorization of n is not known to P it suffices to check that $c, c' \bmod n \neq 0$.

Note that unlike Schnorr-like proof over known-order groups, since it does not know the order of $\mathcal{QR}_n$, the prover masks its witness (a, ρ) using much larger (a', ρ') .

The security of Protocol 4.2, implicit in [CMP20], is stated in the following theorem.

Theorem 4.3 (Security of Protocol 4.2). *If assumption 2.14 holds, then Protocol 4.2 is computational Sigma protocol for $\{\Pi_{s, d_v, d_r, \text{pp}}^{\text{Rng}}\}$ with respect to the distribution ensemble $\{P_{\kappa=(\kappa_s, \kappa_c)}\}$ that outputs $s \leftarrow 2^{\kappa_s + \kappa_c + 1}$, arbitrary $d_v, d_r \in \mathbb{N}$, and $\text{pp} \xleftarrow{R} \text{Gen}(1^{\kappa_s}, 1^{\kappa_c})$.*

That is, assuming strong RSA, with all but negligible probability over pp , Protocol 4.2 is a computational Sigma protocol for $\Pi_{s, d_v, d_r, \text{pp}}^{\text{Rng}}$. We prove Theorem 4.3 by proving that Protocol 4.2 is a (non-computational) Sigma protocol for the following promise problem:

Definition 4.4 (Π^{Rng}). *For $\kappa_c, s, d_v, d_r \in \mathbb{N}$, let*

$$\Pi_{\kappa_c, s, d_v, d_r}^{\text{Rng}} := \begin{cases} \text{For} & z = ((n, g, h), c), (x, r, \gamma) \\ \text{Yes :} & \{z : \text{IntCom}_{\text{pp}}(x; r) = c \wedge x \in [d_v], r \in [d_r]\} \\ \text{Slack :} & \{z : (\text{IntCom}_{\text{pp}}(x; r) = \pm c \wedge x \in [\pm s \cdot d_v]) \\ & \quad \vee \gcd(x, n) \in (1, 2^{\kappa_c + 1}] \\ & \quad \vee (c \in \mathbb{Z}_n^*, \gamma \leq 2^{\kappa_c - 1}) \wedge (g^x \cdot h^r = c^\gamma \bmod n) \wedge \\ & \quad (\gamma \nmid x \vee \gamma \nmid r \vee (g^{x/\gamma} \cdot h^{r/\gamma} \neq \pm c \bmod n))\} \end{cases}.$$

That is, the slack instances not only allow the input to be larger than the designated range, but also enables the extractor to output (c, x, r, γ) that sets f to be true in Proposition 2.16. We sometimes refer to the first condition of the slack set, i.e., $\text{IntCom}_{\text{pp}}(x; r) = \pm c \wedge x \in [\pm d_v \cdot s]$, as the *meaningful slack instances*. We prove the following result.

Theorem 4.5. *There exists PPTM's Sim and Ext such that the following holds for any $p = (\kappa_c, s, d_v, d_r) \in \mathbb{N}^4$: Protocol 4.2 is a Sigma-protocol for Π_p^{Rng} , with simulator Sim_p and POK extractor Ext_p .*

Theorem 4.5 is proved below, but we first use it for proving Theorem 4.3.

Proof of Theorem 4.3. Completeness and zero-knowledge readily follow from Theorem 4.5, which states that these properties holds for *any* choice of pp . For the POK part, we prove that the extractor Ext guaranteed by Theorem 4.5 is a good extractor. Fix parameters $p \leftarrow (1^{\kappa_s}, 1^{\kappa_c}, d_v, d_r, \text{pp})$ and two transcripts $\{t_j \leftarrow (c, c', e_j, \gamma_j, a_j'', \rho_j'')\}_{j \in \{0,1\}}$ that make V accept. Let $s \leftarrow 2^{\kappa_s + \kappa_c + 2}$ and

$(x, \rho, \gamma) \leftarrow \text{Ext}_{s,p}(c, c', e_0, e_1, \gamma_0, \gamma_1)$. Note that if $\gcd(x, n) \in (1, \dots, 2^{\kappa_c+1}]$, then x is a non-trivial factorization of n . Since we assume assumption 2.14 holds, this happens only with negligible probability. Similarly, by Proposition 2.16, it holds that $(c \in \mathbb{Z}_n^*, \gamma \leq 2^{\kappa_c-1}) \wedge (g^x \cdot h^r = c^\gamma \bmod n)$ which only happens with negligible probability over the choice of pp . So with all but negligible probability over the choice of pp , it holds that $\text{IntCom}_{\text{pp}}(x; \rho) = \pm c \wedge x \in [d_v \cdot s]$, concluding the proof. $\square$

Proof of Theorem 4.5. Fix $p = (\kappa_c, s, d_v, d_r) \in \mathbb{N}^3$. The proof follows by Claims 4.6, 4.8 and 4.10 stated below. $\square$

Claim 4.6 (Correctness). *Protocol 4.2 is perfectly correct.*

Proof. Immediate. $\square$

Zero knowledge. Consider the following simulator.

Algorithm 4.7 (Sim).

Parameters: p .

Input: $c \in \mathbb{N}$.

Operation:

1. Sample

(a) $a'' \xleftarrow{\text{R}} [2^{\kappa_s + \kappa_c} \cdot d_v]$.

(b) $\rho'' \xleftarrow{\text{R}} [2^{\kappa_s + \kappa_c} \cdot d_r]$.

(c) $e \xleftarrow{\text{R}} [2^{\kappa_c}]$.

2. Let $c' \leftarrow \text{IntCom}_{\text{pp}}(a''; \rho'') \cdot (c^e)^{-1} \bmod n$.^a

3. Output (c', e, a'', ρ'') .

^a c^e is guaranteed to have an inverse since $c \in \mathbb{Z}_n^*$ for any correct c in the language, since $c = g^a \cdot h^\rho \bmod n$ where $g, h \in \mathbb{Z}_n^*$.

Claim 4.8 (Zero knowledge). *For any $((n, g, h), c), (a, r, \gamma)) \in \text{Yes}_p^{\text{Rng}}$, the output of $\text{Sim}_p(c)$ is $2^{1-\kappa_s}$ -close to the semi-honest execution on parameter p and common input c .*

Proof. Let $a' \leftarrow a'' - a \cdot e$ and $\rho' \leftarrow \rho'' - \rho \cdot e$. For c' computed by by Sim it holds that

$$c' = g^{a''} \cdot h^{\rho''} \cdot g^{-a \cdot e} \cdot h^{-\rho \cdot e} = g^{a'' - a \cdot e} \cdot h^{\rho'' - \rho \cdot e} = g^{a'} \cdot h^{\rho'} \bmod n$$

Hence, the value of c' in the emulated execution and in the real execution is the same function of (c, e, a'', ρ'') . Since, the e 's in both executions are identically distributed, it is left to bound the statistical distance between (a'', ρ'') given c, e .

In the emulated execution, $a'' \xleftarrow{\text{R}} [2^{\kappa_s + \kappa_c} \cdot d_v]$ and $\rho'' \xleftarrow{\text{R}} [2^{\kappa_s + \kappa_c} \cdot d_r]$. In contrast, in the real execution, $a'' = e \cdot a + a'$ and $\rho'' = e \cdot \rho + \rho'$, for $a' \xleftarrow{\text{R}} [2^{\kappa_s + \kappa_c} \cdot d_v]$ and $\rho' \xleftarrow{\text{R}} [2^{\kappa_s + \kappa_c} \cdot d_r]$. It follows that the statistical distance between the value of a'' 's is at most $ea/(2^{\kappa_s + \kappa_c} \cdot d_v) \leq 2^{-\kappa_s}$ and between the value of ρ'' 's is at most $e\rho/(2^{\kappa_s + \kappa_c} \cdot d_r) \leq 2^{-\kappa_s}$. All in all, the statistical distance is at most $2 \cdot 2^{-\kappa_s}$. $\square$

Special soundness. Consider the following knowledge extractor.

Algorithm 4.9 (Ext).

Parameter: p .

Input: $c \in \mathbb{N}$ and two transcripts $(c', e_0, a_0'', \rho_0'')$ and $(c', e_1, a_1'', \rho_1'')$.

Operation:

1. Let $\alpha \leftarrow a_0'' - a_1''$, $\beta \leftarrow \rho_0'' - \rho_1''$ and $\gamma \leftarrow e_0 - e_1$.
2. If the event $E := (\gamma \mid \alpha) \wedge (\gamma \mid \beta) \wedge (g^{\alpha/\gamma} \cdot h^{\beta/\gamma} = \pm c \bmod n)$ is true, output $(\alpha' \leftarrow \alpha/\gamma, \beta' \leftarrow \beta/\gamma)$.
3. Else, output (α, β, γ) .

Claim 4.10 (Special soundness). *For any input $c \in \mathbb{N}$ and two transcript $t_0 = (c', e_0, a_0'', \rho_0'')$ and $t_1 = (c', e_1, a_1'', \rho_1'')$ with $e_0 > e_1$ and $\mathbf{V}_p(c, t_j) = 1$ for both $j \in \{0, 1\}$, it holds that $(c, \text{Ext}_p(c, t_0, t_1)) \in \text{Slack}_p^{\text{Rng}}$.*

Proof. Since both t_0 and t_1 are accepting, for both $j \in \{0, 1\}$:

1. $a_j'' \in [2^{\kappa_s + \kappa_c + 1} \cdot d_v]$.
2. $g^{a_j''} \cdot h^{\rho_j''} = c^{e_j} \cdot c' \bmod n$.

By the second item, we deduce that $g^\alpha \cdot h^\beta = c^\gamma \bmod n$. Now if $E = \text{true}$, then $\text{IntComScpp}(\alpha'; \beta') = g^{\alpha'} \cdot h^{\beta'} = \pm c \bmod n$ and $\alpha \in [\pm d_v \cdot 2^{\kappa_s + \kappa_c + 1}]$. So indeed $(c, \text{Ext}_p(c, t_0, t_1)) \in \text{Slack}_p^{\text{Rng}}$.

Else, observe that $\gamma < 2^{\kappa_c - 1}$ (since each of $e_0, e_1 \in [2^{\kappa_c - 1}]$), $g^\alpha \cdot h^\beta = c^\gamma \bmod n$ (by their definition above), and at least one of the following holds: $\gamma \nmid \alpha$, $\gamma \nmid \beta$ or $g^{\alpha/\gamma} \cdot h^{\beta/\gamma} \neq \pm c \bmod n$. Hence, $(c, \text{Ext}_p(c, t_0, t_1)) \in \text{Slack}_p^{\text{Rng}}$ also in this case. $\square$

5 Applications to Other Additively Homomorphic Commitments

The main application of the integer commitments highly efficient range-proof protocols, as the one described in Section 4, is to provide highly efficient equality and range proofs for different types of homomorphic commitments. An equality proof enables proving that two additively homomorphic commitments open to the same value. The two commitments can be any combination of integer commitments, Pedersen or ElGamal commitments, or the Paillier additively homomorphic public-key encryption scheme (naturally viewed as a commitment). These equality proofs immediately yield efficient range proofs, in the sense of Protocol 4.2, for the above types of additively homomorphic commitments (none of them enjoys native efficient range proofs as integer commitments has).

In Section 5.1, we present equality proofs for different types of homomorphic commitments. In Section 5.2, we explain how to use these equality proofs to get range proof protocols for the different types of homomorphic commitments considered there. Finally, in Section 5.3, we describe several applications of the above equality and range proofs.

5.1 Equality Proofs

We start in Section 5.1.1 by presenting an equality proof between integer commitments and the additively homomorphic “group commitments”, naturally defined by exponentiation over prime-order groups, i.e., $\text{Com}_{\mathbb{G},G}(x) := x \cdot G$.¹¹ In Section 5.1.2, we present an equality proof between integer commitments and Pedersen commitments over prime-order groups, and in Section 5.1.3, we do the same for Paillier encryption. Finally, in Section 5.1.4, we explain how to use the above protocols to obtain equality proofs between any type of commitment (group, Pedersen, or Paillier).

To make the text more readable, in the following we denote integer commitments using an overhead tilde, e.g., $\tilde{A} \leftarrow \text{IntCom}_{\text{pp}}(a; \rho)$.

5.1.1 Equality between Group Commitments in Rough Groups

Definition 5.1. A group $\mathbb{G}$ is ℓ -rough, for $\ell \in \mathbb{N}$, if $\ell' \nmid |\mathbb{G}|$, for every $\ell' < \ell$.

Equality proof for integer and group commitments is a protocol for the following promise problem:

$$\Pi_{s,d_v,d_r,\text{pp},G}^{\text{EqGrp}} := \begin{cases} \text{Yes:} & \left\{ ((\tilde{A}, A), (a, \rho)) : \text{IntCom}_{\text{pp}}(a; \rho) = \tilde{A} \wedge a \cdot G = A \wedge a \in [d_v], \rho \in [d_r] \right\} \\ \text{Slack:} & \left\{ ((\tilde{A}, A), (a, \rho)) : \text{IntCom}_{\text{pp}}(a; \rho) = \pm \tilde{A} \wedge a \cdot G = A \wedge a \in [\pm s \cdot d_v] \right\} \end{cases}$$

Note that the description of the generator G of the additive group is part of the above problem, but as we shall see later, G can be a fixed generator of a “large enough” group. Also note that the above promise problem not only states that a is the opening of the two commitments, but also that it is small. Our protocol critically relies on this bound to guarantee zero-knowledge. Furthermore, this bound is critical for our applications of equality proof.

Protocol 5.2 (Equality proof with a group commitment).

Parties: P, V .

Parameters: $1^{\kappa_s}, 1^{\kappa_c}, d_v, d_r \in \mathbb{N}, \text{pp} = (n, g, h) \in \mathbb{N}^3, G$.

Common input: $\tilde{A}, A$.

P ’s private input: $a, \rho \in \mathbb{N}$.

Operation: The protocol follows the same lines as Protocol 4.2, with the following additions:

Step 1. P (also) sends $A' \leftarrow a' \cdot G$ to V .

Step 4. V (also) verifies that $a'' \cdot G = e \cdot A + A'$.

Theorem 5.3 (Security of Protocol 5.2). *If assumption 2.14 holds, then Protocol 5.2 is computational Sigma protocol for $\left\{ \Pi_{s,d_v,d_r,\text{pp}}^{\text{EqGrp}} \right\}$ with respect to the distribution ensemble $\{P_{\kappa=(\kappa_s, \kappa_c)}\}$ that outputs $s \leftarrow 2^{\kappa_s + \kappa_c + 2}$, arbitrary $d_v, d_r \in \mathbb{N}$, generator G of a 2^{κ_c} -rough group, and $\text{pp} \xleftarrow{R} \text{Gen}(1^{\kappa_s}, 1^{\kappa_c})$.*

Proof.

¹¹Obviously, being deterministic, group commitments are not hiding (at least not in the usual sense), but it is simpler to explain equality proofs using this simple example.

Correctness. Clear.

Zero knowledge. Consider the simulator Sim' that acts like algorithm 4.7 but in Step 3, also output $A' \leftarrow a'' \cdot G - e \cdot A$. It is clear that the added A' part in the view is perfectly emulated by Sim' . Hence, the zero-knowledge property follows from the same lines as that of Protocol 4.2.

Special soundness. Fix two accepting transcripts $\left\{(\tilde{A}', A', e_j, a_j''', \rho_j'')\right\}_{j \in \{0,1\}}$ with $e_0 > e_1$, and let $\alpha \leftarrow a_0''' - a_1'''$, $\beta \leftarrow \rho_0'' - \rho_1''$ and $\gamma \leftarrow e_0 - e_1$. The proof of Theorem 4.3 yields that with all but negligible probability

$$(\gamma \mid \alpha) \wedge (\gamma \mid \beta) \wedge (g^{\alpha/\gamma} \cdot h^{\beta/\gamma} = \pm c \bmod n) \quad (5)$$

We instruct the extractor to output $(\alpha' \leftarrow \alpha/\gamma, \beta' \leftarrow \beta/\gamma)$. Since α' is of the right size, it is thus left to argue that $\alpha' \cdot G = A$. Since both transcripts are accepting, it holds that $x_j'' \cdot G = e_j \cdot A + A'$ for both $j \in \{0,1\}$, and therefore $\gamma \cdot A = \alpha \cdot G$. Let q be the not-necessarily-prime order of the group generated by G . Next, we will be using the following fact.

Fact 5.4. *Let $\ell, q \in \mathbb{N}$ such that $\ell' \nmid q$ for all $\ell' < \ell$. Then, $\gcd(\ell', q) = 1$, for all $\ell' < \ell$.*

Proof. Fix $\ell' < \ell$ and let $p_1^{a_1} \cdots p_k^{a_k}$ denote the prime factorization of ℓ' . By assumption, for all $i \in [k]$, $p_i \nmid q$, and thus $\gcd(p_i, q) = 1$, for all $i \in [k]$. Consequently, $\gcd(\ell', q) = 1$. $\square$

Since $\mathbb{G}$ is 2^{κ_c} -rough and $\gamma \in [2^{\kappa_c} - 1]$, it follows by fact 5.4 that $\gcd(\gamma, q) = 1$ and therefore $A = \gamma^{-1} \cdot \alpha \cdot G$. Writing $\alpha = \alpha' \cdot \gamma$, we deduce that $\gamma^{-1} \cdot \alpha = \alpha' \bmod q$, and therefore $A = \alpha' \cdot G$. $\square$

5.1.2 Equality with Pedersen Commitments in Known-Order Rough Groups

In this section, we present an equality proof protocol between integer commitments and the (additively homomorphic) Pedersen commitments over known-order rough groups, e.g., elliptic curve groups. The following text readily extends to *ElGamal* commitments over the same types of groups.

For a generator G of an additive group $\mathbb{G}$, the Pedersen commitment with respect to public key $E \in \mathbb{G}$, input $m \in \mathbb{Z}_q$, and randomness $\rho \in \mathbb{Z}_q$, is defined by $\text{Ped}_{G,E}(m; \rho) := \rho \cdot E + m \cdot G$. That is, the Pedersen commitment is just an integer commitment written here in additive form, where the operations are performed in the group. Consider the following promise problem:

$$\Pi_{s,d_v,d_r,\text{pp},G,E}^{\text{EqPed}} := \begin{cases} \text{Yes :} & \{((\tilde{A}, \hat{A}), (a, \rho, \hat{\rho})) : \text{IntCom}_{\text{pp}}(a; \rho) = \tilde{A} \wedge \text{Ped}_{G,E}(a; \hat{\rho}) = \hat{A} \wedge a \in [d_v], \rho \in [d_r]\} \\ \text{Slack :} & \{((\tilde{A}, \hat{A}), (a, \rho, \hat{\rho})) : \text{IntCom}_{\text{pp}}(a; \rho) = \pm \tilde{A} \wedge \text{Ped}_{G,E}(a; \hat{\rho}) = \hat{A} \wedge a \in [\pm s \cdot d_v]\} \end{cases}$$

Namely, a Yes instance is a pair of Integer and Pedersen commitments to the same (not too large) value.

Protocol 5.5 (Equality proof with Pedersen Commitments).

Parties: P, V.

Parameters: $1^{\kappa_s}, 1^{\kappa_c}$, $d_v, d_r \in \mathbb{N}$, $\text{pp} = (n, g, h) \in \mathbb{N}^3$, q, G, E .

Common input: $\tilde{A}, \hat{A}$.

P's private input: $a, \rho, \hat{\rho} \in \mathbb{N}$.

Operation: The protocol follows the same lines as Protocol 4.2, with the following additions:

Step 1. P (also) sends $\hat{A}' \leftarrow \text{Ped}_{G,E}(a'; \tilde{\rho}')$ to V, for $\tilde{\rho}' \xleftarrow{R} \mathbb{Z}_q$.

Step 3. P (also) sends $\tilde{\rho}'' \leftarrow e \cdot \hat{\rho} + \tilde{\rho}' \bmod q$ to V.

Step 4. V (also) verifies that $\text{Ped}_{G,E}(a''; \tilde{\rho}'') = e \cdot \hat{A} + \hat{A}'$.

Theorem 5.6 (Security of Protocol 5.5). *If assumption 2.14 holds, then Protocol 5.5 is computational Sigma protocol for $\{\Pi_{s,d_v,d_r,\text{pp},E}^{\text{EqPed}}\}$ with respect to the distribution ensemble $\{P_{\kappa=(\kappa_s,\kappa_c)}\}$ that outputs $s \leftarrow 2^{\kappa_s+\kappa_c+2}$, arbitrary $d_v, d_r \in \mathbb{N}$, G that is a generator of an additive q -order 2^{κ_c} -rough group $\mathbb{G}$, $E \in \mathbb{G}$, and $\text{pp} \xleftarrow{R} \text{Gen}(1^{\kappa_s}, 1^{\kappa_c})$.*

Proof.

Correctness. Clear.

Zero knowledge. Consider the simulator Sim' that acts like algorithm 4.7 but adds the following values to the generated view:

- $\tilde{\rho}'' \xleftarrow{R} \mathbb{Z}_q$.
- $\hat{A}' \leftarrow \text{Ped}_{G,E}(a''; \tilde{\rho}'') - e \cdot \hat{A}$.

It is clear that the added $(\hat{A}', \tilde{\rho}'')$ part in the view, is perfectly emulated by Sim'. Hence, the zero-knowledge property follows from the same lines as that of Protocol 4.2.

Special soundness. Fix two accepting transcripts $\{(\tilde{A}', \hat{A}', e_j, a_j'', \rho_j'', \tilde{\rho}_j'')\}_{j \in \{0,1\}}$ with $e_0 > e_1$, and let $\alpha \leftarrow a_0''' - a_1'''$, $\beta \leftarrow \rho_0'' - \rho_1''$, $\hat{\beta} \leftarrow \tilde{\rho}_0'' - \tilde{\rho}_1''$ and $\gamma \leftarrow e_0 - e_1$. The proof of Theorem 4.3 yields that with all but negligible probability

$$(\gamma \mid \alpha) \wedge (\gamma \mid \beta) \wedge (g^{\alpha/\gamma} \cdot h^{\beta/\gamma} = \pm c \bmod n) \quad (6)$$

Note that since $\gamma \in [2^{\kappa_c} - 1]$, $\gcd(\gamma, q) = 1$. We instruct the extractor to output $(\alpha' \leftarrow \alpha/\gamma, \beta' \leftarrow \beta/\gamma, \hat{\beta}' \leftarrow \hat{\beta} \cdot \gamma^{-1} \bmod q)$. It is thus left to prove that

$$\hat{A} = \text{Ped}_{G,E}(\alpha'; \gamma^{-1} \cdot \hat{\beta})$$

Since both transcripts are accepting, we get that $\hat{A} = \text{Ped}_{G,E}(\gamma^{-1} \cdot \alpha \bmod q; \gamma^{-1} \cdot \hat{\beta})$. This concludes the proof since, as in the proof of Theorem 5.3 we have that $\gamma^{-1} \cdot \alpha = \alpha' \bmod q$. $\square$

5.1.3 Equality with Paillier Encryption

In this section, we present an equality proof protocol between integer commitments and the additively homomorphic Paillier encryption scheme introduced by Paillier [Pai99]. Recall that for a public key $n \in \mathbb{N}$, the plaintext domain is $\mathbb{Z}_n^*$, and the randomness domain is $\mathbb{Z}_n^*$. Given

$m \in \mathbb{Z}_{n^2}^*$ and randomness $\rho \in \mathbb{Z}_n^*$, the Paillier encryption with respect to public key n is defined by $\text{PEnc}_n(m; \rho) := (n+1)^m \cdot \rho^n \bmod n^2$. Consider the following promise problem.

$$\Pi_{\mathbf{s}, d_v, d_r, \mathbf{pp}, \mathbf{pk}}^{\text{EqPal}} := \begin{cases} \text{Yes :} & \{((\tilde{A}, \hat{A}), (a, \rho, \hat{\rho})) : \\ & \text{IntCom}_{\mathbf{pp}}(a; \rho) = \tilde{A} \wedge \text{PEnc}_{\mathbf{pk}}(a; \hat{\rho}) = \hat{A} \wedge a \in [d_v], \rho \in [d_r]\} \\ \text{Slack :} & \{((\tilde{A}, \hat{A}), (a, \rho, \hat{\rho})) : \\ & \text{IntCom}_{\mathbf{pp}}(a; \rho) = \pm \tilde{A} \wedge \text{PEnc}_{\mathbf{pk}}(a; \hat{\rho}) = \hat{A} \wedge a \in [\pm \mathbf{s} \cdot d_v]\} \end{cases}$$

Namely, a Yes instance is a pair of Integer commitment and a Paillier encryption of the same (not too large) value.

Protocol 5.7 (Equality proof with Paillier encryption).

Parties: P, V.

Parameters: $1^{\kappa_s}, 1^{\kappa_c}, d_v, d_r, \mathbf{pp} = (n, g, h) \in \mathbb{N}^3, \mathbf{pk} \in \mathbb{N}$.

Common input: $\tilde{A}, \hat{A}$.

P's private input: $a, \rho, \hat{\rho} \in \mathbb{N}$.

Operation: The protocol follows the same lines as Protocol 4.2, with the following additions:

Step 1. P (also) sends $\hat{A}' \leftarrow \text{PEnc}_{\mathbf{pk}}(a'; \hat{\rho}')$ to V, for $\hat{\rho}' \xleftarrow{R} \mathbb{Z}_{\mathbf{pk}}^*$.

Step 3. P (also) sends $\hat{\rho}'' \leftarrow \hat{\rho}^e \cdot \hat{\rho}' \bmod \mathbf{pk}$ to V.

Step 4. V (also) verifies that $\text{PEnc}_{\mathbf{pk}}(a''; \hat{\rho}'') = e \cdot \hat{A} + \hat{A}'$ and that $\hat{\rho}'' \in \mathbb{Z}_{\mathbf{pk}}^*$.

Let PKeyGen be any algorithm that on input 1^{κ_c} outputs $\mathbf{pk} \in \mathbb{N}$ whose prime factors are of size at least 2^{κ_c} .¹²

Theorem 5.8 (Security of Protocol 5.7). *If assumption 2.14 holds and factoring is hard with respect to PKeyGen, then Protocol 5.7 is computational Sigma protocol for $\{\Pi_{\mathbf{s}, d_v, d_r, \mathbf{pp}, \mathbf{pk}}^{\text{EqPal}}\}$ with respect to the distribution ensemble $\{P_{\kappa=(\kappa_s, \kappa_c)}\}$ that outputs $\mathbf{s} \leftarrow 2^{\kappa_s + \kappa_c + 2}$, arbitrary $d_v, d_r \in \mathbb{N}$, $\mathbf{pp} \xleftarrow{R} \text{Gen}(1^{\kappa_s}, 1^{\kappa_c})$ and $\mathbf{pk} \xleftarrow{R} \text{PKeyGen}(1^{\kappa_c})$.*

In the proof of Theorem 5.8, we use the following fact about the “Paillier modulus”.

Paillier ciphertexts divisors.

Proposition 5.9 (Paillier ciphertexts divisors). *There exists a PPTM that given as input $n \in \mathbb{N}$, $x \in \mathbb{Z}_n$, $z, \rho \in \mathbb{Z}_n^*$ and $\tilde{A} \in \mathbb{Z}_{n^2}^*$ such that $\text{PEnc}_n(z \cdot x; \rho) = z \cdot \tilde{A}$, it outputs $r \in \mathbb{Z}_n^*$ with $\text{PEnc}_n(x; r) = \tilde{A}$.*

Proposition 5.9 is proved using the following fact.

Proposition 5.10. *There exists a PPTM that given as input $n \in \mathbb{N}$ and $\rho, y, z \in \mathbb{N}$ such that*

¹²This formulation allows the prover to sample $\mathbf{pk}$ by itself, and prove that it is of the right form. Note that we do not require $\gcd(\mathbf{pk}, \phi(n)) = 1$, which is not needed for the range proof.

1. ρ, z are relatively prime to n , and

2. $\rho^n = y^z \bmod n^2$,

it outputs $r \in \mathbb{Z}_n^*$ with $r^n = y \bmod n^2$.

Proof. Using the extended GCD algorithm, we find the Bezout coefficients $u, v \in \mathbb{Z}$ such that $u \cdot z + v \cdot n^2 = 1$. Let $r \leftarrow \rho^u \cdot y^{nv} \bmod n^2$. It holds that

$$r^n = \rho^{un} \cdot y^{vn^2} = y^{zu} \cdot y^{vn^2} = y \bmod n^2.$$

Note that the same holds for $\rho' \leftarrow r \bmod n$ (powering by n removes any additive terms that are divisible by n). Finally, since ρ and y are relatively prime to n , so are r and ρ' . $\square$

Proof of Proposition 5.9. For $\hat{Y} \leftarrow \tilde{A} - \text{PEnc}_n(x; 1)$, it holds that $z \cdot \hat{Y} = \text{PEnc}_n(0; \rho)$. Thus, for $y \leftarrow \hat{Y}$ (y is viewed as integer), it holds that $y^z = \rho^n \bmod n^2$. By Proposition 5.10, we can efficiently compute $r \in \mathbb{Z}_n^*$ such that $r^n = y \bmod n^2$. It follows that $\text{PEnc}_n(0; r) = \hat{Y}$, and therefore $\text{PEnc}_{\text{pk}}(x; r) = \tilde{A}$. $\square$

Proving Theorem 5.8. Using Proposition 5.9, we prove Theorem 5.8 as follows.

Proof of Theorem 5.8.

Correctness. Clear.

Zero knowledge. Consider the simulator Sim' that acts like algorithm 4.7 but adds the following values to the generated view:

- $\hat{\rho}'' \xleftarrow{\mathcal{R}} \mathbb{Z}_{\text{pk}}^*$.
- $\hat{A}' \leftarrow \text{PEnc}_{\text{pk}}(a''; \hat{\rho}'') - e \cdot \hat{A}$.

It is clear that the added $(\hat{A}', \hat{\rho}'')$ part in the view, is perfectly emulated by Sim' . Hence, the zero-knowledge property follows from the same lines as that of Protocol 4.2.

Special soundness. Fix two accepting transcripts $\left\{ (\tilde{A}', \hat{A}', e_j, a_j''', \rho_j'', \hat{\rho}_j'') \right\}_{j \in \{0,1\}}$ with $e_0 > e_1$, and let $\alpha \leftarrow a_0''' - a_1'''$, $\beta \leftarrow \rho_0'' - \rho_1''$, $\hat{\beta} \leftarrow \hat{\rho}_0'' \cdot (\hat{\rho}_1'')^{-1} \bmod \text{pk}$ (note that $\hat{\rho}_1'' \in \mathbb{Z}_{\text{pk}}^*$), and $\gamma \leftarrow e_0 - e_1$. The proof of Theorem 4.3 yields that with all but negligible probability

$$(\gamma \mid \alpha) \wedge (\gamma \mid \beta) \wedge (g^{\alpha/\gamma} \cdot h^{\beta/\gamma} = \pm c \bmod n) \quad (7)$$

Since both transcripts are accepting, it holds that

$$\text{PEnc}_{\text{pk}}(\alpha; \hat{\beta}) = \gamma \cdot \tilde{A} \quad (8)$$

Since $\gamma \in [2^{\kappa_c} - 1]$, it holds that $\gcd(\gamma, \text{pk}) = 1$. Thus by Proposition 5.9 (letting $n \leftarrow \text{pk}$, $x \leftarrow \alpha'$, $z \leftarrow \gamma$, and $\rho \leftarrow \hat{\beta}$), we can efficiently compute $\hat{\beta}' \in \mathbb{Z}_{\text{pk}}^*$ such that $\text{PEnc}_{\text{pk}}(\alpha'; \hat{\beta}') = \tilde{A}$. Outputting $(\alpha', \beta', \hat{\beta}')$ concludes the proof. $\square$

5.1.4 Equality between Different Homomorphic Commitments

The above protocols can be used for equality proofs between any combination of homomorphic commitment schemes. For concreteness, below we focus on commitments of two different rough groups, but the following easily extends to any combination of commitments in rough groups, Pedersen commitments and Paillier encryptions.

Specifically, an easy application of Protocol 5.2 is a computational, extended, Sigma protocol for the following promise problem:

$$\Pi_{s, d_v, G, \hat{G}}^{\text{EqGrp}} := \begin{cases} \text{Yes :} & \left\{ ((A, \hat{A}), a) : a \cdot G = A \wedge a \cdot \hat{G} = \hat{A} \wedge a \in [d_v] \right\} \\ \text{Slack :} & \left\{ ((A, \hat{A}), a) : a \cdot G = A \wedge a \cdot \hat{G} = \hat{A} \wedge a \in [\pm s \cdot d_v] \right\} \end{cases}$$

where G and $\hat{G}$ are generators of two suitable rough groups. Note that without the bound on a , the above promise problem is meaningless: let q and $\hat{q}$ be the orders of the prime order groups generated by G and $\hat{G}$, respectively, and assume $\gcd(q, \hat{q}) = 1$. By CRT, for any $a \in \mathbb{Z}_q$ and $\hat{a} \in \mathbb{Z}_{\hat{q}}$ there exists $b \in \mathbb{Z}_{q \cdot \hat{q}}$ such that $b = a \bmod q$ and $b = \hat{a} \bmod \hat{q}$. So *any* $(A, \hat{A})$ is a Yes instance for this variant. In typical applications, we set the bound d_v so that $q > s \cdot d_v$. Hence, $(A, \hat{A})$ is a Yes instance only if the *unique* discrete log $a \in [q - 1]$ of A , is a discrete log of $\hat{A}$.

The protocol is straightforward. In the preamble stage, the verifier samples pp using Gen and proves to the prover that it is well formed. Then the prover sends $\tilde{A} \leftarrow \text{IntCom}_{\text{pp}}(a; \rho)$ to the verifier, and the parties interact in two parallel executions of Protocol 5.2: one with input $(\tilde{A}, A)$ with respect to the group $\mathbb{G}$, and the second with input $(\tilde{A}, \hat{A})$ with respect to the group $\hat{\mathbb{G}}$. (Actually, it is easy to see that it suffices for the verifier to send a single challenge e , to be used for both executions.)

5.2 Range Proofs

The equality proofs presented in Section 5.1 imply range-proof protocols for any of the homomorphic commitment schemes considered there, i.e., rough groups, Pedersen commitments and Paillier encryptions. For concreteness, we focus on rough groups. For such groups, Protocol 5.2 implies an extended computational Sigma protocol for the promise problem

$$\Pi_{s, d_v, G}^{\text{Rng}} = \begin{cases} \text{Yes :} & \{(A, a) : a \cdot G = A \wedge a \in [d_v]\} \\ \text{Slack :} & \{(A, a) : a \cdot G = A \wedge a \in [s \cdot d_v]\} \end{cases}$$

The protocol is straightforward. In the preamble stage, the verifier samples pp using Gen and proves to the prover that it is well formed (see Section 3.2). Then the prover sends $\tilde{A} \leftarrow \text{IntCom}_{\text{pp}}(a; \rho)$ to the verifier, and the parties continue according to Protocol 5.2.

5.3 Applications

In this section we list two important applications of the above equality and range proof protocols.

5.3.1 Oblivious Linear Evaluation

Oblivious Linear Evaluation (OLE), Naor and Pinkas [NP06], is a natural algebraic generalization of *oblivious transfer (OT)* in which the sender supplies the secret coefficients a and b of a line

$a \cdot X + b$ (defined over some finite ring $\mathcal{R}$), and the receiver learns the evaluation of this line on a chosen secret point x . OLE can be used to perform multiplication of secrets in $\mathcal{R}$ in a manner analogous to the way in which OT is used to perform secure multiplication of boolean secrets (i.e., “secure AND”) [GMW87; Kil88], i.e., in $\mathbb{Z}_2$. This makes OLE a useful building block in a large variety of applications, including threshold cryptography [Doe+18; Hai+18; Doe+24] and secure computation of general arithmetic circuits [IPS09; KOS16].

Since Paillier homomorphic commitment is actually a public encryption scheme, it naturally admits a straightforward OLE protocol.¹³ This simple OLE can be bootstrapped to obtain an OLE in the case where the parties’ inputs are committed using other homomorphic commitment schemes, by using an equality proof between the commitment and the Paillier ciphertext [HL09]. The above approach has a long history in constructions of general purpose secure protocols [CDN01; Ben+11; DO10], and has more recently seen particular popularity in the threshold ECDSA literature [Lin17; GG18; Can+20; Adj+24] as well as in widely deployed threshold ECDSA systems for digital asset custody [Coi25; Fir23].

5.3.2 Commit&Modular-Reveal

In the *commit-ℰmodular-reveal* functionality, the sender first commits to integer $a \in \mathbb{Z}_n$, and later reveals $a \bmod r$ for r chosen by the receiver. The commit&modular-reveal functionality was recently used in Doerner et al. [Doe+25] for constructing highly-efficient (OT-based) OLE protocols. They presented two implementations of this functionality. A rather inefficient OT-based implementation, and a highly efficient integer-commitment-based implementation that we discuss here.

Implementing commit&modular-reveal using integer-commitment is straightforward. The sender commits, using integer commitment, to a , and proves that $a \in \mathbb{Z}_n$ using a range proof protocol.¹⁴ After receiving r , the sender sends $a_r \leftarrow a \bmod r$ to the receiver, and uses Protocol 5.2 with group $\mathbb{G}$ taken to be the additive group $\mathbb{Z}_r$, i.e., the additive group generated by 1 modulo r , to prove that $a = a_r \bmod r$. If r does not have small factors, the resulting protocol is secure against semi-honest receivers (recall that the zero-knowledge property of Protocols 4.2 and 5.2 is only guaranteed to hold against honest verifiers). Yet, the protocol can be compiled using the Fiat-Shamir paradigm, to get a fully secure protocol in the random-oracle model. See [Doe+25] for details.

6 Lightweight Leaky Well-Formedness Proof

A main obstacle when one considers using integer commitments is the sampling of the parameters $\mathbf{pp} = (n, g, h)$. Recall that for the soundness of the range and equality proofs, it is required that $\mathbf{pp}$ is uniformly sampled without leaking the randomness used to sample it to the prover. For the hiding property to hold, however, it is required that $g \in \langle h \rangle_n$. An immediate solution would be to let the verifier sample $\mathbf{pp}$ and prove that $g \in \langle h \rangle_n$, but unfortunately no highly efficient proof for this task is known. The following highly-efficient sigma protocol, to be used by the verifier,

¹³This technique is folklore—see e.g., [Ben+11] for a formal treatment. In a nutshell, using any additively homomorphic encryption scheme, the receiver sends $\text{enc}(x)$ for its input x under a public key for which it holds the decryption key. The sender then operates homomorphically on the ciphertext to compute $\text{enc}(ax + b)$, where a and b are the sender’s inputs, and sends the result to the receiver. The receiver outputs the decryption of the received ciphertext.

¹⁴If it uses range proof with slack, the guarantee on the size of a will also be up to the slackness guarantee.

does not enforce $g \in \langle h \rangle_n$, but it does guarantee that any leakage induced by committing with parameters that have passed this verification is limited. In many settings, like those in this paper, this leakage can be easily overcome, admitting the use of integer commitments with a light setup phase.

Protocol 6.1 (Parameters well-formedness).

Parties: P, V.

Parameters: 1^{κ_s} .

Common input: $n, g, h \in \mathbb{N}$.

P's private input: $\alpha \in \mathbb{N}$.

Operation:

1. P:
 - (a) Sample $\beta \xleftarrow{R} [n \cdot 2^{2\kappa_s}]$.
 - (b) Send $c \leftarrow h^\beta \bmod n$ to V.
2. V: Send $e \xleftarrow{R} [2^{\kappa_s}]$ to P.
3. P: Send $x \leftarrow e \cdot \alpha + \beta$ to V.
4. V:
 - (a) Verify $h^x = c \cdot g^e \bmod n$.
 - (b) Verify $h \in \mathbb{Z}_n^*$.

It is clear that Protocol 6.1 is correct and zero-knowledge for the relation

$$\mathcal{R}^{\text{WF}} := \{((n, g, h), \alpha) : g = h^\alpha \bmod n \wedge \alpha \in [n]\}.$$

Claim 6.2. *Protocol 6.1 is perfectly correct and $(2^{-\kappa_s})$ -statistical honest-verifier zero-knowledge for $\mathcal{R}^{\text{WF}}$.*

Proof. Correctness holds by construction. The zero-knowledge simulator

1. Samples $e \xleftarrow{R} [2^{\kappa_s}]$ and $x \xleftarrow{R} [n \cdot 2^{2\kappa_s}]$.
2. Sets $c \leftarrow h^x \cdot (g^e)^{-1} \bmod n$.
3. Outputs (c, e, x) .

It is clear that this simulation is at most $2^{-\kappa_s}$ -far from the honest execution. □

The soundness of the protocol is given in the following claim.

Definition 6.3. *For $n, h, g \in \mathbb{N}$, let $\delta_{n,h}(g) := \min\{i \in \mathbb{N} : g^i \in \langle h \rangle\}$.*

That is, $\delta_{n,h}(g)$ is the smallest non-zero integer i such that $g^i = h^j$, for some $j \in \mathbb{N}$.

Claim 6.4. *On common input $n, g, h \in \mathbb{N}$, V accepts in Protocol 6.1 with probability at most $2^{-\kappa_s} + 1/\delta_{n,h}(g)$.*

Specifically, if V accepts with probability greater than $2 \cdot 2^{-\kappa_s}$, then there exists $i \leq 2^{\kappa_s}$ such that $g^i \in \langle h \rangle_n$.

Proof of Claim 6.4. Special case of Theorem 7.6. □

The following claim yields that if $\delta_{n,h}(g)$ is small (and then V might accept), using $\mathbf{pp} = (n, g, h)$ still yields a “not-too-leaky” commitment.

Claim 6.5. *Let $\mathbf{pp} = (1^{\kappa_s}, n, h, g)$ and let $i \in \mathbb{N}$ be such that $g^i \in \langle h \rangle_n$. Then*

$$\text{SD}(\text{IntCmt}_{\mathbf{pp}}(1^{\kappa_s}, x), \text{IntCmt}_{\mathbf{pp}}(1^{\kappa_s}, x')) \leq 2^{-\kappa_s}$$

for any $x, x' \in \mathbb{Z}$ with $x = x' \bmod i$.

Namely, a commitment to x only leaks $x \bmod \delta_{n,h}(g)$.

Proof. Write $x = x_0 + i \cdot x_1$ for $x_0 = x \bmod i$ and define x'_0 and x'_1 analogously with respect to x' . Let $\hat{g} \leftarrow g^i$ and $\mathbf{pp}' = (n, h, \hat{g})$. It holds that

$$\text{IntCom}_{\mathbf{pp}}(x; \rho) = h^\rho \cdot g^x = h^\rho \cdot g^{x_0} \cdot \hat{g}^{x_1} = \text{IntCom}_{\mathbf{pp}'}(x_1; \rho) \cdot g^{x_0} \bmod n.$$

Since $\hat{g} \in \langle h \rangle_n$, it holds that $\text{IntCmt}_{\mathbf{pp}'}$ is hiding. Additionally, since $g^{x_0} = g^{x'_0} \bmod n$, we conclude that $\text{IntCmt}_{\mathbf{pp}}(1^{\kappa_s}, x)$ is of statistical distance at most $2^{-\kappa_s}$ from $\text{IntCmt}_{\mathbf{pp}}(1^{\kappa_s}, x')$. □

6.1 Applications

In this section we describe some applications in which using Protocol 6.1 for generating the integer parameters provides good enough security.

6.1.1 Commitments over Rough Integer Rings

When the value to commit comes from $\mathbb{Z}_q$ for some $q \in \mathbb{N}$ that does not admit small factors, Protocol 6.1 can be used to obtain a non-leaky commitment; after using the light-setup protocol (yielding parameter $\mathbf{pp}$), commit to $x \in \mathbb{Z}_q$ as follows:

Algorithm 6.6 (IntCom').

Parameters: $\mathbf{pp} = (1^{\kappa_s}, n, g, h), q$.

Input: x .

Operation:

1. Sample $x' \xleftarrow{\mathbb{R}} [q \cdot 2^{2\kappa_s}]$ conditioned on $x' = x \bmod q$.
2. Output $\text{IntCom}_{\mathbf{pp}}(x'; \rho)$ for $\rho \xleftarrow{\mathbb{R}} [n \cdot 2^{\kappa_s}]$.

Claim 6.7. *Let $\mathbf{pp} = (1^{\kappa_s}, n, g, h)$ and q such that $\min\{d \in \mathbb{N} \setminus \{1\} : d \mid q\} \geq 2^{\kappa_s}$ and $\delta_{n,h}(g) \leq 2^{\kappa_s}$. Then for any $x_0, x_1 \in \mathbb{Z}_q$, the statistical distance between $\text{IntCom}'(x_0)$ and $\text{IntCom}'(x_1)$ is at most $2^{-\kappa_s}$.*

That is, either the light-setup protocol aborts with probability $1 - 2^{-\kappa_s}$, or IntCom' is statistically hiding.

Proof. Let $\delta \leftarrow \delta_{n,h}(g)$, $s \leftarrow q \cdot 2^{2\kappa_s}$ and $s' \leftarrow q \cdot \delta$ that is divisible by δ . If IntCom' were to sample $x' \xleftarrow{R} [s']$, then, given $\gcd(\delta, q) = 1$ as $\delta \leq 2^{\kappa_s}$ and q does not admit small non-trivial factors, by CRT, $\text{IntCom}'(x_0)$ and $\text{IntCom}'(x_1)$ would be identically distributed (given pp). It follows that in the real algorithm, $\text{IntCom}'(x_0)$ and $\text{IntCom}'(x_1)$ are at distance at most $s'/s \leq \delta/2^{2\kappa_s} \leq 2^{-\kappa_s}$. $\square$

Recall, see Section 3.2, that the zero knowledge property of all the integer commitment Sigma protocols considered in this paper hold for *any* choice (well formed or not) of the parameter pp . Hence, for values in $\mathbb{Z}_q$ (for any suitable q), using Protocol 6.1 yields a highly efficient well-formedness proof for integer commitment-based protocols.

6.1.2 Oblivious Linear Evaluation

Using leaky integer commitments in a Paillier-based OLE has a few subtleties. If the OLE is performed over a small prime-order field, or over an integer ring whose modulus is divisible by a small prime, then the construction is insecure: the integer commitment leaks information about the parties' inputs—recall that the sender holds inputs a, b and the receiver holds x —and this leakage is non-simulatable. However, if the modulus has no small prime factors (i.e., the group is rough), then noise can be added to the inputs while preserving their residuosity class modulo the target modulus, and the construction is secure. This idea has been used for instance in [Adj+24], and the security argument is essentially the same as the one described for the commit&modular-reveal protocol describe next.

6.1.3 Commit & Modular Reveal

Recall that in the commit&modular-reveal functionality, the sender first commits to integer $a \in \mathbb{Z}_n$, and later reveals $a \bmod r$ for r chosen by the receiver. Consider the variant of the protocol described in Section 5.3.2 when using Protocol 6.1 for choosing the integer commitment parameters. By Claims 6.4 and 6.5, the resulting protocol might leak $a \bmod i$ to the receiver for some $i \in [2^{\kappa_s}]$. When using the commit-&modular-reveal functionality, however, we already have to incorporate the, by design, leakage of $a \bmod r$ to the receiver. The latter is handled by sampling a random $a \xleftarrow{R} [n]$ conditioned on $a = a_q \bmod q$ for some has-to-be-kept-secret $a_q \in \mathbb{Z}_q$ where q is prime. By the Chinese Remainder Theorem, if $r < q$ and n is 2^{κ_s} larger than $q \cdot r$, then the leakage $a \bmod r$ is insignificant. Thus, for $q > 2^{\kappa_s}$, by choosing n that is 2^{κ_s} larger than $q \cdot r \cdot 2^{\kappa_s}$, the above leaky implementation of the commit-&modular-reveal functionality is good enough. See [Doe+25] for details.

7 Vector Commitments

In this section, we define the natural generalization to vector integer commitments, which allows a committer to publish a single element $c \in \mathbb{Z}_n^*$ while committing to *many* integers $x_1, \dots, x_k$. In the following, for $\mathbf{g} = (g_1, \dots, g_k)$ let $\text{IntVecCom}_{n,\mathbf{g},h}(\mathbf{x}, \rho) := g_1^{x_1} \cdots g_k^{x_k} \cdot h^\rho \bmod n$. Since essentially all properties and proofs carry over from the singleton setting with only minor changes, we present only brief intuition and proof sketches for the basic properties. We instead focus on the lightweight

leaky well-formedness property, which constitutes our main new contribution and for which we give a full, rigorous security proof.

Algorithm 7.1 (Vector integer commitment). The integer vector commitment scheme $\text{IntVecComSc} = (\text{Gen}, \text{Commit}, \text{Verify})$ is defined as follows:

Gen: On input $1^{\kappa_s}, 1^{\kappa_c}$ and $k, s \in \mathbb{N} \cup \{0\}$ ($s \leftarrow 0$ if not set):

1. $n \xleftarrow{R} \text{StModuliGen}(1^{\kappa_c}, s)$.
2. Sample $h \xleftarrow{R} \mathcal{QR}_n$ and $\alpha_1, \dots, \alpha_k \xleftarrow{R} [n \cdot 2^{\kappa_s}]$.
3. Let $g_i \leftarrow h^{\alpha_i} \bmod n$.
4. Output $\text{pp} \leftarrow (\kappa \leftarrow (\kappa_s, \kappa_c), n, \mathbf{g} \leftarrow (g_1, \dots, g_k), h)$.

Let ExtGen be the variant of **Gen** that also outputs $\alpha \leftarrow (\alpha_1, \dots, \alpha_k)$.

Commit: On input $\text{pp} = (\kappa, n, \mathbf{g}, h)$ and $\mathbf{m} \leftarrow (m_1, \dots, m_k) \in \mathbb{Z}^k$:

1. $\rho \xleftarrow{R} [n \cdot 2^{\kappa_s}]$.
2. Output $\text{IntVecCom}_{n, \mathbf{g}, h}(\mathbf{m}; \rho)$.

Verify: On input $\text{pp} = (\kappa, n, \mathbf{g}, h)$, $c \in \mathbb{Z}_n$ and $\mathbf{m}, \rho \in \mathbb{Z}$:

Output true iff $\text{IntVecCom}_{n, \mathbf{g}, h}(\mathbf{m}; \rho) = \pm c \bmod n$.

Recall that the parameter s enforces StModuliGen to choose $n > s$. When clear from the context, we typically omit κ from pp .

Correctness, statistical hiding & computational binding. Correctness is perfect and statistical hiding follows trivially from the singleton case with $2^{-\kappa_s}$ statistical distinguishing advantage (exactly as in the singleton case).

The binding property reduces to the hardness of factoring, and the reduction is as tight as in the singleton (non-vector) case. In particular, if an adversary breaks binding, then—except with probability $2^{-\kappa_s}$ —the reduction outputs a nonzero multiple of the order of h .

Range proof. Extending the range proof soundness guarantee to the vector case boils down to generalizing Proposition 2.15 in showing that it is infeasible for an attacker to generate $c \in \mathbb{Z}_n^*$ and $\alpha_1, \dots, \alpha_k, \beta, \gamma$ such that $\prod_{i=1}^k g_i^{\alpha_i} \cdot h^\beta = c^\gamma$ where $\gamma \nmid \beta$ or $\gamma \nmid \alpha_i$ for some $i \in [k]$. By following the same steps as in the proof of Proposition 2.15, if $\gamma \nmid t \leftarrow \beta + \sum_{i=1}^k \lambda_i \alpha_i$, where each λ_i is the discrete logarithm (chosen by the reduction) of g_i with respect to h , then the simulator breaks strong RSA by computing a non-trivial root of h . Finally, over the choice of the λ_i 's, if $\gamma \nmid \beta$ or $\gamma \nmid \alpha_i$ for some $i \in [k]$, then $\gamma \mid t$ with probability at most $1/2$.

The correctness and honest verifier zero-knowledge properties are immediate.

7.1 Lightweight Leaky Well-Formedness Proof for Vector Commitments

To prove well-formedness of the vector commitment parameters, instead of proving that each g_i lies in $\langle h \rangle$ individually, we use a batching trick to handle all statements with a single proof. Specifically,

the prover computes the discrete log of a random linear combination of the g_i 's — a task that becomes intractable if any g_i is far from $\langle h \rangle$.

Protocol 7.2 (Parameters well-formedness).

Parties: P, V.

Parameters: 1^{κ_s} .

Common input: $n, h \in \mathbb{N}, \mathbf{g} = (g_1, \dots, g_k) \in N^k$.

P's private input: $\alpha_1, \dots, \alpha_k \in \mathbb{N}$.

Operation:

1. P:
 - (a) Sample $\beta \xleftarrow{R} [n \cdot 2^{2\kappa_s}]$.
 - (b) Send $c \leftarrow h^\beta \pmod n$ to V.
2. V: Send $(e_1, \dots, e_k) \xleftarrow{R} [2^{\kappa_s}]^k$ to P.
3. P: Send $x \leftarrow \beta + \sum_{i=1}^k e_i \cdot \alpha_i$ to V.
4. V:
 - (a) Verify $h^x = c \cdot \prod_{i=1}^k g_i^{e_i} \pmod n$.
 - (b) Verify $h \in \mathbb{Z}_n^*$ and $\mathbf{g} \in (\mathbb{Z}_n^*)^k$.

It is clear that Protocol 7.2 is correct and zero-knowledge for the relation

$$\mathcal{R}^{\text{WF}} := \left\{ ((n, \mathbf{g}, h), \alpha) : g_i = h^{\alpha_i} \pmod n \wedge \alpha \in [n]^k \right\}.$$

Claim 7.3. *Protocol 7.2 is perfectly correct and $(k \cdot 2^{-\kappa_s})$ -statistical honest-verifier zero-knowledge for $\mathcal{R}^{\text{WF}}$.*

Proof. Correctness holds by construction. The zero-knowledge simulator is defined as follows:

Algorithm 7.4 (S).

1. Sample $(e_1, \dots, e_k) \xleftarrow{R} [2^{\kappa_s}]^k$ and $x \xleftarrow{R} [n \cdot 2^{2\kappa_s}]$.
2. Set $c \leftarrow h^x \cdot (\prod_{i=1}^k g_i^{e_i})^{-1} \pmod n$.
3. Output $(c, e_1, \dots, e_k, x)$.

It is clear that the output of S is at most $k \cdot 2^{-\kappa_s}$ -far from the honest execution. □

The soundness of the protocol is given in the following claim. Recall that $\text{ord}_n(a) := |\langle a \rangle_n|$.

Definition 7.5. *For $n, h \in \mathbb{N}$ and $\mathbf{g} \in \mathbb{N}^k$, define*

$$\delta_{n,h}(\mathbf{g}) := \max \left\{ \min \{ i \in \mathbb{N} : g_\ell^i \in \langle h \rangle \} : \ell \in [k] \right\}.$$

That is, $\delta_{n,h}(\mathbf{g})$ is the largest value among the minimal integers $i_1, \dots, i_k$ satisfying $g_\ell^{i_\ell} \in \langle h \rangle_n$.

Theorem 7.6. *On common input $n, h \in \mathbb{N}$, and $\mathbf{g} \in \mathbb{N}^k$, $\mathcal{V}$ accepts in Protocol 7.2 with probability at most $2^{-\kappa_s} + 1/\delta_{n,h}(\mathbf{g})$.*

Proof of Theorem 7.6. Without loss of generality, assume that $g_1^{\delta_{n,h}(\mathbf{g})} \in \langle h \rangle_n$ and $g_1^i \notin \langle h \rangle_n$ for $i < \delta_{n,h}(\mathbf{g})$, and let $\delta \leftarrow \delta_{n,h}(\mathbf{g})$. We show that for any suitable $c, e_2, \dots, e_k$,

$$\Pr_{e_1 \xleftarrow{\mathbb{R}} [2^{\kappa_s}]} \left[c \cdot \prod_{i=1}^k g_i^{e_i} \in \langle h \rangle_n \right] \leq \frac{1}{2^{\kappa_s}} + \frac{1}{\delta_{n,h}(\mathbf{g})} \quad (9)$$

Write $u = c^{-1} \prod_{i=2}^k g_i^{-e_i} \bmod n$,¹⁵ and let $H = \langle h \rangle_n$, $G = \langle g_1, h \rangle_n$. In the quotient group G/H ,¹⁶ write $\bar{g}_1 = g_1 \cdot H$ and $\bar{u} = u \cdot H$. We observe that Equation (9) is equivalent to

$$\Pr_{e_1 \xleftarrow{\mathbb{R}} [2^{\kappa_s}]} [\bar{g}_1^{e_1} = \bar{u}] \leq \frac{1}{2^{\kappa_s}} + \frac{1}{\delta} \quad (10)$$

By noting that the order of $\bar{g}_1$ is exactly δ , we observe that $\bar{g}_1^e = \bar{u}$ implies $\bar{g}_1^{e+\delta} = \bar{u}$ and $\bar{g}_1^{e+j} \neq \bar{u}$ for all $j \in \{1, \dots, \delta - 1\}$. Therefore, there are at most $\lfloor 2^{\kappa_s}/\delta \rfloor + 1$ consecutive points with step δ in the interval $[2^{\kappa_s}]$. This concludes the proof. $\square$

Remark 7.7 (Tightness of Theorem 7.6). It is worth mentioning that the guarantee of Theorem 7.6 is essentially the best one can hope for in practical applications. Although it does not capture the adversary's cheating probability in all cases, it is the natural worst-case guarantee and it is infeasible to distinguish this worst case from executions in which the adversary's actual cheating probability is smaller than what Theorem 7.6 guarantees. For instance, if $\langle g_1 H, g_2 H \rangle \cong \langle g_1 H \rangle \times \langle g_2 H \rangle$, then one can show that the cheating probability is roughly $1/|\langle g_1 H, g_2 H \rangle|$, rather than $\max\{1/|\langle g_1 H \rangle|, 1/|\langle g_2 H \rangle|\}$. However, it is infeasible to distinguish between the case where $\delta \leftarrow |\langle g_1 H, g_2 H \rangle|$ and $\langle g_1 H, g_2 H \rangle \cong \langle g_1 H \rangle \times \langle g_2 H \rangle$, and the case where $\delta = |\langle g_1 H \rangle|$ and $\langle g_2 H \rangle = \{H\}$. Consequently, the same amount of noise must be added to the secrets to guarantee hiding in either case, even though, in principle, the former case could allow for less noise.

8 Historical notes

RSA-based integer commitments were first introduced by Fujisaki and Okamoto [FO97], which also introduced the range proof protocol. However, the security proof of Fujisaki and Okamoto [FO97] contained an error, which was later fixed by Damgård and Fujisaki [DF02]. Damgård and Fujisaki [DF02] also describe a generalization to not-necessarily-safe-prime RSA moduli, as well as other hidden-order groups, such as class groups. The intervening work of Boudot [Bou00] can be seen as building on [FO97] and as a precursor to Damgård and Fujisaki [DF02].

In terms of applications, equality between homomorphic commitments and encryption first appeared in the context of distributed DSA in [MR01], which claims to prove equality between a

¹⁵We know that c and $g_2, \dots, g_k$ are invertible because the verifier checks that $h, g_1, \dots, g_k \in \mathbb{Z}_n^*$ and $h^z = c \prod_{i=1}^k g_i^{e_i}$.

¹⁶For two arbitrary abelian groups S and T such that T is a subgroup of S , the quotient group S/T is the set of cosets $\{s \cdot T = \{st : t \in T\} : s \in S\}$ endowed with the group operation $sT \cdot s'T = ss'T$ and identity element T .

Paillier ciphertext, an elliptic curve commitment, and an integer commitment—essentially combining the Paillier equality proof and the rough-group equality proof from this tutorial. These proofs were used extensively in the distributed ECDSA literature [GG18; GG20; LN18; CMP20; Can+20], but did not account for the subtleties introduced by a maliciously chosen Paillier public key, which allows breaking soundness and has led to key extraction attacks on some of these protocols [MYG24]. The soundness gap was patched in [CMP20]. Additionally, the commit-and-prove application was further used in [Doe+25] to show that a committed integer has a given residue class modulo some prime—a proof that appears essential for the malicious security of their OLE protocol.

Finally, we mention the recent work [Adj+24], in which several improvements to integer commitments are considered under more aggressive assumptions—specifically different classes of moduli instead of safe-prime moduli, and small exponent optimizations.

References

- [Adj+24] Michael Adjedj, Constantin Blokh, Geoffroy Couteau, Arik Galansky, Antoine Joux, and Nikolaos Makriyannis. *Two-Round 2PC ECDSA at the Cost of 1 OLE*. Cryptology ePrint Archive, Report 2024/1950. 2024. URL: <https://eprint.iacr.org/2024/1950> (cit. on pp. 28, 31, 35).
- [Ben+11] Rikke Bendlin, Ivan Damgård, Claudio Orlandi, and Sarah Zakarias. “Semi-homomorphic Encryption and Multiparty Computation”. In: *EUROCRYPT 2011*. Ed. by Kenneth G. Paterson. Vol. 6632. LNCS. Springer, Berlin, Heidelberg, May 2011, pp. 169–188. DOI: [10.1007/978-3-642-20465-4_11](https://doi.org/10.1007/978-3-642-20465-4_11) (cit. on p. 28).
- [Bon02] Dan Boneh. “Twenty Years of Attacks on the RSA Cryptosystem”. In: *NOTICES OF THE AMS* 46 (Feb. 2002) (cit. on p. 16).
- [Bou00] Fabrice Boudot. “Efficient Proofs that a Committed Number Lies in an Interval”. In: *EUROCRYPT 2000*. Ed. by Bart Preneel. Vol. 1807. LNCS. Springer, Berlin, Heidelberg, May 2000, pp. 431–444. DOI: [10.1007/3-540-45539-6_31](https://doi.org/10.1007/3-540-45539-6_31) (cit. on pp. 5, 34).
- [Bri+88] Ernest F. Brickell, David Chaum, Ivan Damgård, and Jeroen van de Graaf. “Gradual and Verifiable Release of a Secret”. In: *CRYPTO’87*. Ed. by Carl Pomerance. Vol. 293. LNCS. Springer, Berlin, Heidelberg, Aug. 1988, pp. 156–166. DOI: [10.1007/3-540-48184-2_11](https://doi.org/10.1007/3-540-48184-2_11) (cit. on p. 5).
- [Bün+18] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. “Bulletproofs: Short Proofs for Confidential Transactions and More”. In: *2018 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, May 2018, pp. 315–334. DOI: [10.1109/SP.2018.00020](https://doi.org/10.1109/SP.2018.00020) (cit. on p. 5).
- [Can+20] Ran Canetti, Rosario Gennaro, Steven Goldfeder, Nikolaos Makriyannis, and Udi Peled. “UC Non-Interactive, Proactive, Threshold ECDSA with Identifiable Aborts”. In: *ACM CCS 2020*. Ed. by Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna. ACM Press, Nov. 2020, pp. 1769–1787. DOI: [10.1145/3372297.3423367](https://doi.org/10.1145/3372297.3423367) (cit. on pp. 3, 5, 28, 35).

- [CCs08] Jan Camenisch, Rafik Chaabouni, and abhi shelat. “Efficient Protocols for Set Membership and Range Proofs”. In: *ASIACRYPT 2008*. Ed. by Josef Pieprzyk. Vol. 5350. LNCS. Springer, Berlin, Heidelberg, Dec. 2008, pp. 234–252. DOI: [10.1007/978-3-540-89255-7_15](https://doi.org/10.1007/978-3-540-89255-7_15) (cit. on p. 5).
- [CDN01] Ronald Cramer, Ivan Damgård, and Jesper Buus Nielsen. “Multiparty Computation from Threshold Homomorphic Encryption”. In: *EUROCRYPT 2001*. Ed. by Birgit Pfitzmann. Vol. 2045. LNCS. Springer, Berlin, Heidelberg, May 2001, pp. 280–299. DOI: [10.1007/3-540-44987-6_18](https://doi.org/10.1007/3-540-44987-6_18) (cit. on p. 28).
- [Chr+24] Miranda Christ, Foteini Baldimtsi, Konstantinos Kryptos Chalkias, Deepak Maram, Arnab Roy, and Joy Wang. *SoK: Zero-Knowledge Range Proofs*. Cryptology ePrint Archive, Report 2024/430. 2024. URL: <https://eprint.iacr.org/2024/430> (cit. on p. 5).
- [CMP20] Ran Canetti, Nikolaos Makriyannis, and Udi Peled. *UC Non-Interactive, Proactive, Threshold ECDSA*. Cryptology ePrint Archive, Report 2020/492. 2020. URL: <https://eprint.iacr.org/2020/492> (cit. on pp. 12, 13, 19, 35).
- [Coi25] Coinbase. *Coinbase Open Source MPC Library*. GitHub repository. Accessed: March 17, 2026. 2025. URL: <https://github.com/coinbase/cb-mpc> (cit. on p. 28).
- [Cou+21] Geoffroy Couteau, Michael Klooß, Huang Lin, and Michael Reichle. “Efficient Range Proofs with Transparent Setup from Bounded Integer Commitments”. In: *EUROCRYPT 2021, Part III*. Ed. by Anne Canteaut and François-Xavier Standaert. Vol. 12698. LNCS. Springer, Cham, Oct. 2021, pp. 247–277. DOI: [10.1007/978-3-030-77883-5_9](https://doi.org/10.1007/978-3-030-77883-5_9) (cit. on pp. 4, 5).
- [Cou+22] Geoffroy Couteau, Dahmun Goudarzi, Michael Klooß, and Michael Reichle. “Sharp: Short Relaxed Range Proofs”. In: *ACM CCS 2022*. Ed. by Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi. ACM Press, Nov. 2022, pp. 609–622. DOI: [10.1145/3548606.3560628](https://doi.org/10.1145/3548606.3560628) (cit. on p. 5).
- [CPP17] Geoffroy Couteau, Thomas Peters, and David Pointcheval. “Removing the Strong RSA Assumption from Arguments over the Integers”. In: *EUROCRYPT 2017, Part II*. Ed. by Jean-Sébastien Coron and Jesper Buus Nielsen. Vol. 10211. LNCS. Springer, Cham, 2017, pp. 321–350. DOI: [10.1007/978-3-319-56614-6_11](https://doi.org/10.1007/978-3-319-56614-6_11) (cit. on p. 5).
- [Dam10] Ivan Damgård. *On Sigma-protocols*. Manuscript. 2010. URL: <https://www.cs.au.dk/ivan/Sigma.pdf> (cit. on p. 6).
- [DF02] Ivan Damgård and Eiichiro Fujisaki. “A Statistically-Hiding Integer Commitment Scheme Based on Groups with Hidden Order”. In: *ASIACRYPT 2002*. Ed. by Yuliang Zheng. Vol. 2501. LNCS. Springer, Berlin, Heidelberg, Dec. 2002, pp. 125–142. DOI: [10.1007/3-540-36178-2_8](https://doi.org/10.1007/3-540-36178-2_8) (cit. on pp. 3, 5, 34).
- [DO10] Ivan Damgård and Claudio Orlandi. “Multiparty Computation for Dishonest Majority: From Passive to Active Security at Low Cost”. In: *CRYPTO 2010*. Ed. by Tal Rabin. Vol. 6223. LNCS. Springer, Berlin, Heidelberg, Aug. 2010, pp. 558–576. DOI: [10.1007/978-3-642-14623-7_30](https://doi.org/10.1007/978-3-642-14623-7_30) (cit. on p. 28).

- [Doe+18] Jack Doerner, Yashvanth Kondi, Eysa Lee, and abhi shelat. “Secure Two-party Threshold ECDSA from ECDSA Assumptions”. In: *2018 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, May 2018, pp. 980–997. DOI: [10.1109/SP.2018.00036](https://doi.org/10.1109/SP.2018.00036) (cit. on p. 28).
- [Doe+24] Jack Doerner, Yashvanth Kondi, Eysa Lee, and abhi shelat. “Threshold ECDSA in Three Rounds”. In: *2024 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, May 2024, pp. 3053–3071. DOI: [10.1109/SP54263.2024.00178](https://doi.org/10.1109/SP54263.2024.00178) (cit. on p. 28).
- [Doe+25] Jack Doerner, Iftach Haitner, Yuval Ishai, and Nikolaos Makriyannis. “From OT to OLE with Subquadratic Communication”. In: *ACM CCS 2025*. Ed. by Chun-Ying Huang, Jyh-Cheng Chen, Shih-Pyng Shieh, David Lie, and Véronique Cortier. ACM Press, Oct. 2025, pp. 2249–2263. DOI: [10.1145/3719027.3765225](https://doi.org/10.1145/3719027.3765225) (cit. on pp. 28, 31, 35).
- [Fir23] Fireblocks. *Fireblocks MPC Library*. GitHub repository. Accessed: March 17, 2026. 2023. URL: <https://github.com/fireblocks/mpc-lib> (cit. on p. 28).
- [FO97] Eiichiro Fujisaki and Tatsuaki Okamoto. “Statistical Zero Knowledge Protocols to Prove Modular Polynomial Relations”. In: *CRYPTO’97*. Ed. by Burton S. Kaliski Jr. Vol. 1294. LNCS. Springer, Berlin, Heidelberg, Aug. 1997, pp. 16–30. DOI: [10.1007/BFb0052225](https://doi.org/10.1007/BFb0052225) (cit. on pp. 3, 5, 12, 34).
- [GG18] Rosario Gennaro and Steven Goldfeder. “Fast Multiparty Threshold ECDSA with Fast Trustless Setup”. In: *ACM CCS 2018*. Ed. by David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang. ACM Press, Oct. 2018, pp. 1179–1194. DOI: [10.1145/3243734.3243859](https://doi.org/10.1145/3243734.3243859) (cit. on pp. 3, 5, 28, 35).
- [GG20] Rosario Gennaro and Steven Goldfeder. *One Round Threshold ECDSA with Identifiable Abort*. Cryptology ePrint Archive, Report 2020/540. 2020. URL: <https://eprint.iacr.org/2020/540> (cit. on p. 35).
- [GMW87] Oded Goldreich, Silvio Micali, and Avi Wigderson. “How to Play any Mental Game or A Completeness Theorem for Protocols with Honest Majority”. In: *19th ACM STOC*. Ed. by Alfred Aho. ACM Press, May 1987, pp. 218–229. DOI: [10.1145/28395.28420](https://doi.org/10.1145/28395.28420) (cit. on p. 28).
- [Gro05] Jens Groth. “Non-interactive Zero-Knowledge Arguments for Voting”. In: *ACNS 2005*. Ed. by John Ioannidis, Angelos Keromytis, and Moti Yung. Vol. 3531. LNCS. Springer, Berlin, Heidelberg, June 2005, pp. 467–482. DOI: [10.1007/11496137_32](https://doi.org/10.1007/11496137_32) (cit. on p. 5).
- [Gro11] Jens Groth. “Efficient Zero-Knowledge Arguments from Two-Tiered Homomorphic Commitments”. In: *ASIACRYPT 2011*. Ed. by Dong Hoon Lee and Xiaoyun Wang. Vol. 7073. LNCS. Springer, Berlin, Heidelberg, Dec. 2011, pp. 431–448. DOI: [10.1007/978-3-642-25385-0_23](https://doi.org/10.1007/978-3-642-25385-0_23) (cit. on p. 5).
- [Gro16] Jens Groth. “On the Size of Pairing-Based Non-interactive Arguments”. In: *EUROCRYPT 2016, Part II*. Ed. by Marc Fischlin and Jean-Sébastien Coron. Vol. 9666. LNCS. Springer, Berlin, Heidelberg, May 2016, pp. 305–326. DOI: [10.1007/978-3-662-49896-5_11](https://doi.org/10.1007/978-3-662-49896-5_11) (cit. on p. 5).

- [Hai+18] Iftach Haitner, Yehuda Lindell, Ariel Nof, and Samuel Ranellucci. *Fast Secure Multiparty ECDSA with Practical Distributed Key Generation and Applications to Cryptocurrency Custody*. Cryptology ePrint Archive, Paper 2018/987. 2018. DOI: [10.1145/3243734.3243788](https://eprint.iacr.org/2018/987). URL: <https://eprint.iacr.org/2018/987> (cit. on p. 28).
- [HL09] Carmit Hazay and Yehuda Lindell. *Efficient Oblivious Polynomial Evaluation with Simulation-Based Security*. Cryptology ePrint Archive, Report 2009/459. 2009. URL: <https://eprint.iacr.org/2009/459> (cit. on p. 28).
- [IPS09] Yuval Ishai, Manoj Prabhakaran, and Amit Sahai. “Secure Arithmetic Computation with No Honest Majority”. In: *TCC 2009*. Ed. by Omer Reingold. Vol. 5444. LNCS. Springer, Berlin, Heidelberg, Mar. 2009, pp. 294–314. DOI: [10.1007/978-3-642-00457-5_18](https://doi.org/10.1007/978-3-642-00457-5_18) (cit. on p. 28).
- [Kil88] Joe Kilian. “Founding Cryptography on Oblivious Transfer”. In: *20th ACM STOC*. ACM Press, May 1988, pp. 20–31. DOI: [10.1145/62212.62215](https://doi.org/10.1145/62212.62215) (cit. on p. 28).
- [KOS16] Marcel Keller, Emmanuela Orsini, and Peter Scholl. “MASCOT: Faster Malicious Arithmetic Secure Computation with Oblivious Transfer”. In: *ACM CCS 2016*. Ed. by Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi. ACM Press, Oct. 2016, pp. 830–842. DOI: [10.1145/2976749.2978357](https://doi.org/10.1145/2976749.2978357) (cit. on p. 28).
- [Lin17] Yehuda Lindell. “Fast Secure Two-Party ECDSA Signing”. In: *CRYPTO 2017, Part II*. Ed. by Jonathan Katz and Hovav Shacham. Vol. 10402. LNCS. Springer, Cham, Aug. 2017, pp. 613–644. DOI: [10.1007/978-3-319-63715-0_21](https://doi.org/10.1007/978-3-319-63715-0_21) (cit. on pp. 5, 28).
- [Lip03] Helger Lipmaa. “On Diophantine Complexity and Statistical Zero-Knowledge Arguments”. In: *ASIACRYPT 2003*. Ed. by Chi-Sung Lai. Vol. 2894. LNCS. Springer, Berlin, Heidelberg, 2003, pp. 398–415. DOI: [10.1007/978-3-540-40061-5_26](https://doi.org/10.1007/978-3-540-40061-5_26) (cit. on p. 5).
- [LN18] Yehuda Lindell and Ariel Nof. “Fast Secure Multiparty ECDSA with Practical Distributed Key Generation and Applications to Cryptocurrency Custody”. In: *ACM CCS 2018*. Ed. by David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang. ACM Press, Oct. 2018, pp. 1837–1854. DOI: [10.1145/3243734.3243788](https://doi.org/10.1145/3243734.3243788) (cit. on pp. 5, 35).
- [Mil76] Gary L. Miller. “Riemann’s hypothesis and tests for primality”. In: *Journal of Computer and System Sciences* 13.3 (1976), pp. 300–317. URL: <https://www.sciencedirect.com/science/article/pii/S0022000076800438> (cit. on p. 16).
- [MR01] Philip D. MacKenzie and Michael K. Reiter. “Two-Party Generation of DSA Signatures”. In: *CRYPTO 2001*. Ed. by Joe Kilian. Vol. 2139. LNCS. Springer, Berlin, Heidelberg, Aug. 2001, pp. 137–154. DOI: [10.1007/3-540-44647-8_8](https://doi.org/10.1007/3-540-44647-8_8) (cit. on pp. 12, 34).
- [MYG24] Nikolaos Makriyannis, Oren Yomtov, and Arik Galansky. “Practical Key-Extraction Attacks in Leading MPC Wallets”. In: *ACM CCS 2024*. Ed. by Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie. ACM Press, Oct. 2024, pp. 3053–3064. DOI: [10.1145/3658644.3670359](https://doi.org/10.1145/3658644.3670359) (cit. on p. 35).

- [NP06] Moni Naor and Benny Pinkas. “Oblivious Polynomial Evaluation”. In: *SIAM Journal on Computing* 35.5 (2006), pp. 1254–1281. DOI: [10.1137/S0097539704383633](https://doi.org/10.1137/S0097539704383633) (cit. on p. 27).
- [Pai99] Pascal Paillier. “Public-Key Cryptosystems Based on Composite Degree Residuosity Classes”. In: *EUROCRYPT’99*. Ed. by Jacques Stern. Vol. 1592. LNCS. Springer, Berlin, Heidelberg, May 1999, pp. 223–238. DOI: [10.1007/3-540-48910-X_16](https://doi.org/10.1007/3-540-48910-X_16) (cit. on p. 24).